



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:
«Здесь пишем тему»

Студент _____
(Группа)

(Подпись, дата)

(И.О. Фамилия)

Руководитель ВКР

(Подпись, дата)

(И.О. Фамилия)

Консультант

(Подпись, дата)

(И.О. Фамилия)

Консультант

(Подпись, дата)

(И.О. Фамилия)

Нормоконтролер

(Подпись, дата)

(И.О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Исследовательская часть	6
1.1 Интуиционистская логика	6
ВНК-интерпретация	7
Семантика Крипке	8
1.2 Системы вывода	10
Система Гильберта	10
Натуральный вывод	11
Секвенциальное исчисление	12
1.3 Типизированное λ -исчисление	14
Аннотирование правил LJ (импликативный фрагмент)	15
Суперскрипты для произведения и суммы	16
Аннотирование правил LJ (полный фрагмент)	17
2 Конструкторская часть	18
2.1 Исчисление LJT	18
Исчисление LJ в формулировке Дайкхоффа	18
Проблема завершаемости	19
Правила исчисления LJT	19
Обратный поиск вывода	21
Пример вывода	21
2.2 Исчисление LJT_{SAT}	22
Клаузная форма	22
Правила исчисления LJT_{SAT}	23
Техника CEGAR	24
Процедура поиска вывода	25
Построение контрмодели Крипке	26
2.3 Аннотирование термами	27
Аннотирование правил LJT	27
Клауфикация как секвенциальные правила	28
Извлечение терма из клаузной формы	30

2.4	Исчисление $C^{\rightarrow}$	31
	Правила исчисления	31
	Аннотирование правил	32
	Процедура поиска вывода	33
	Преимущества перед LJT_{SAT}	34
3	Технологическая часть	35
3.1	Выбранные технологии	35
3.2	Основные структуры данных и классы	35
3.3	Конвейер доказательства	43
	Клаузификация	44
	Поиск в исчислении $C^{\rightarrow}$	46
	Построение LJТ-поддеревьев	47
	Аннотирование λ -термами	53
	Редукция и визуализация	54
3.4	Построение контрмодели	55
3.5	Примеры использования	57
4	Аналитическая часть	61
4.1	Методика измерений	61
4.2	Семейство бенчмарков <i>syj201</i>	62
4.3	Результаты для разработанного доказателя	62
4.4	Сравнение с <i>intuitR</i>	63
	Предположения о расхождении производительности	65
4.5	Семейство бенчмарков <i>syj207</i>	66
4.6	Семейство формул <i>fresh-tail</i>	67
	ЗАКЛЮЧЕНИЕ	70
	СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	72
	ПРИЛОЖЕНИЕ А	73
	ПРИЛОЖЕНИЕ Б	74
	ПРИЛОЖЕНИЕ В	75

ПРИЛОЖЕНИЕ Г	77
ПРИЛОЖЕНИЕ Д	79
ПРИЛОЖЕНИЕ Е	80

ВВЕДЕНИЕ

Изоморфизм Карри-Говарда устанавливает удивительное соответствие между системами формальной логики и вычислительными. Все начинается с наблюдения о том, что импликация $A \rightarrow B$ соответствует типу функции принимающей значение типа A и возвращающей значение типа B . Так как вывод формулы B из посылок $A \rightarrow B$ и A может рассматриваться как применение первой посылки ко второй, точно так будто функция из A в B примененная к A возвращает B .

Данная идея – сопоставление логических формул и некоторых вычислительных выражений – является довольно богатой идеей. Благодаря ней развивается очень перспективная сфера - формальная верификация программ. Формальная верификация позволяет провалидировать (верифицировать) критически важные программы (или их компоненты), например, в ПАО «Группа Астра», занимающейся разработкой ОС Astra Linux, модуль РАМ (управление привелигированным доступом) должен быть формально верифицирован, т.е. должна быть доказана корректность как самих алгоритмов так и их реализации.

Изучение изоморфизма Карри-Говарда сопряжено с существенными трудностями. Прежде всего, сложность обусловлена необходимостью одновременного понимания нескольких абстрактных областей: математической логики (в частности, интуиционистской логики), λ -исчисления и типовых систем. Каждая из этих областей сама по себе требует серьезной теоретической подготовки, а их объединение в рамках единого соответствия «доказательства $\leftrightarrow$ программы» и «формулы $\leftrightarrow$ типы» создает высокий порог входа. Дополнительную сложность вносит высокий уровень абстракции: многие соответствия не имеют наглядной интерпретации и воспринимаются исключительно символически, что затрудняет формирование интуитивного понимания.

В связи с этим разработка специализированного визуализатора представляется особенно актуальной. Создание системы, позволяющей интерактивно отображать соответствия между логическими доказательствами и программами, может существенно повысить доступность данной темы, упростить процесс обучения и способствовать более глубокому пониманию фундаментальных принципов, лежащих в основе современных типовых систем и методов формальной верификации.

1 Исследовательская часть

1.1 Интуиционистская логика

Интуиционистская логика возникла в контексте кризиса оснований математики — масштабного пересмотра фундаментальных допущений, на которых строилось математическое знание. Во второй половине XIX века Георг Кантор заложил основы теории множеств, предложив рассматривать бесконечные совокупности как полноправные математические объекты. Теория Кантора позволила единообразно описать числовые системы, точки континуума и функциональные пространства, однако вскоре обнаружились серьёзные затруднения: в 1897 году Чезаре Бурали-Форти указал на парадокс, связанный с множеством всех ординалов, а в 1901 году Бертран Рассел сформулировал знаменитый парадокс о множестве всех множеств, не содержащих самих себя. Эти противоречия показали, что наивное обращение с бесконечными объектами чревато антиномиями и что сама система допущений классической математики нуждается в критическом анализе.

В ответ на кризис сформировались несколько программ обоснования математики. Давид Гильберт предложил формалистический подход: зафиксировать аксиомы и правила вывода, а затем доказать непротиворечивость полученной формальной системы финитными средствами. Готлоб Фреге и Бертран Рассел развивали логицистскую программу, стремясь свести математику к логике. Третий путь избрал нидерландский математик Луитзен Эгбертус Ян Брауэр: его интуиционизм [1], оформившийся в 1907–1912 годах, предполагал радикальный пересмотр не только оснований, но и самих методов математического рассуждения.

Брауэр исходил из того, что математические объекты не существуют независимо от познающего субъекта, а являются мысленными конструкциями. Математическое утверждение считается истинным лишь тогда, когда для него предъявлена конструкция — явная процедура, позволяющая убедиться в его справедливости. Следствием этой позиции стало отвержение закона исключённого третьего (*tertium non datur*): высказывание $A \vee \neg A$ не может приниматься как логический закон, поскольку для произвольного A мы можем не располагать ни доказательством A , ни доказательством $\neg A$. Классическая логика, напротив, постулирует

этот закон безусловно, допуская тем самым неконструктивные доказательства существования: утверждение $\exists x. P(x)$ считается доказанным, даже если конкретный x не предъявлен, а лишь показано, что отсутствие такого x ведёт к противоречию.

Именно неконструктивность являлась главной мишенью критики Брауэра. Классическое доказательство может гарантировать существование объекта, не указывая ни способа его построения, ни алгоритма нахождения. Для Брауэра такое доказательство лишено математического содержания: оно не даёт конструкции и потому не может считаться обоснованием истинности. Отказ от закона исключённого третьего и, как следствие, от снятия двойного отрицания ($\neg\neg A \rightarrow A$), закона Пирса ($((A \rightarrow B) \rightarrow A) \rightarrow A$) и ряда других классически допустимых принципов позволил ограничить доказательства лишь теми, которые несут конструктивное содержание.

Формальная аксиоматизация интуиционистской логики была осуществлена учеником Брауэра — Арендом Гейтингом [2]. Гейтинг предложил исчисление высказываний, в котором каждая аксиома и правило вывода допускают конструктивную интерпретацию, а закон исключённого третьего не выводим.

ВНК-интерпретация

В 1930–1945 годах Андрей Николаевич Колмогоров и независимо Гейтинг [2] сформулировали интерпретацию логических связок через задачи и решения, известную как ВНК-интерпретация (Брауэр – Гейтинг – Колмогоров). Семантика связок в интуиционистской логике определяется через понятие доказательства:

- **Конъюнкция** $A \wedge B$ считается доказанной, если имеются доказательства как A , так и B ;
- **Дизъюнкция** $A \vee B$ доказана, если известно, какая из частей истинна, и имеется её доказательство;
- **Импликация** $A \rightarrow B$ интерпретируется как конструкция (метод), преобразующая любое доказательство A в доказательство B ;
- **Отрицание** $\neg A$ определяется как $A \rightarrow \perp$, где $\perp$ обозначает ложь;
- **Ложь** $\perp$ не имеет доказательства.

ВНК-интерпретация придала интуиционистской логике ясный вычислительный смысл и послужила одной из отправных точек для изоморфизма Карри–Говарда, устанавливающего соответствие между доказательствами и программами.

Семантика Крипке

Семантика Крипке [3] задаёт класс моделей, в которых истинность формулы определяется относительно миров, связанных отношением частичного порядка. Формула называется общезначаимой, если она истинна в корне любой модели. Контрмодель — модель, в корне которой некоторая формула не вынуждена; существование контрмодели показывает, что формула не является теоремой интуиционистской логики.

Моделью Крипке называется четвёрка $(W, w_0, \preceq, V)$, где

- W — непустое конечное множество миров;
- $w_0 \in W$ — выделенный корневой мир, из которого достигим любой другой: $w_0 \preceq w$ для всех $w \in W$;
- $\preceq$ — частичный порядок (рефлексивное, антисимметричное и транзитивное отношение) на W ;
- V — оценка, сопоставляющая каждой пропозициональной переменной p множество миров $V(p) \subseteq W$, в которых p истинна; оценка удовлетворяет условию наследуемости (монотонности): если $w \in V(p)$ и $w \preceq v$, то $v \in V(p)$.

Отношение вынуждения $w \Vdash A$ (« A истинна в w ») определяется индуктивно:

- $w \Vdash p$ тогда и только тогда, когда $w \in V(p)$;
- $w \Vdash A \wedge B$ тогда и только тогда, когда $w \Vdash A$ и $w \Vdash B$;
- $w \Vdash A \vee B$ тогда и только тогда, когда $w \Vdash A$ или $w \Vdash B$;
- $w \Vdash A \rightarrow B$ тогда и только тогда, когда для всякого $v \succeq w$ из $v \Vdash A$ следует $v \Vdash B$;
- $w \Vdash \perp$ никогда не выполняется.

Тогда $w \Vdash \neg A$ эквивалентно тому, что для всех $v \succeq w$ имеем $v \nVdash A$. Из определения следует монотонность вынуждения: если $w \Vdash A$ и $w \preceq v$, то $v \Vdash A$. Формула в интуиционистской логике общезначаима, если $w_0 \Vdash A$ в любой модели и любой оценке

Пример 1. Рассмотрим цепочку из двух миров $w_0 \prec w_1$ и одну переменную p , положив $V(p) = \{w_1\}$ (наследуемость соблюдена).

Тогда $w_1 \Vdash p$, а $w_0 \nVdash p$.

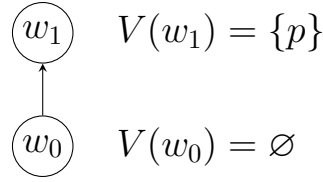


Рисунок 1 — Двухмировая модель Крипке для переменной p

Закон исключённого третьего $p \vee \neg p$. В мире w_0 не вынуждена ни p (нет вынуждения в w_0), ни $\neg p$ (так как $w_1 \succeq w_0$ и $w_1 \Vdash p$). Следовательно, $w_0 \nVdash p \vee \neg p$. Данная модель является контрмоделью, и закон исключённого третьего не общезначим в интуиционистской логике.

Схема снятия двойного отрицания $\neg\neg p \rightarrow p$. Та же модель служит контрмоделью. Имеем $w_0 \nVdash p$. При этом $w_0 \Vdash \neg\neg p$: для любого $v \succeq w_0$, если бы $v \Vdash \neg p$, то для всех $u \succeq v$ было бы $u \nVdash p$, что неверно при $v = w_1$. Значит, $v \nVdash \neg p$ для всех $v \succeq w_0$, откуда $w_0 \Vdash \neg\neg p$. Итак, $w_0 \Vdash \neg\neg p$, но $w_0 \nVdash p$, поэтому $w_0 \nVdash \neg\neg p \rightarrow p$.

Пример 2. Закон Пирса $((p \rightarrow q) \rightarrow p) \rightarrow p$. Возьмём ту же цепочку $w_0 \prec w_1$, но с двумя переменными: $V(p) = \{w_1\}$, $V(q) = \emptyset$.

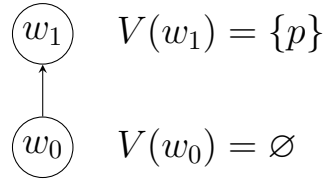


Рисунок 2 — Двухмировая модель Крипке для схемы Пирса

Тогда $w_1 \nVdash q$, откуда $w_0 \nVdash p \rightarrow q$: для $v = w_1$ имеем $v \Vdash p$, но $v \nVdash q$. Проверим $w_0 \Vdash (p \rightarrow q) \rightarrow p$: для любого $v \succeq w_0$ нужно «если $v \Vdash p \rightarrow q$, то $v \Vdash p$ ». При $v = w_0$ посылка $w_0 \Vdash p \rightarrow q$ ложна, импликация выполняется. При $v = w_1$ посылка $w_1 \Vdash p \rightarrow q$ требует, чтобы для всех $u \succeq w_1$ из $u \Vdash p$ следовало $u \Vdash q$; при $u = w_1$ это даёт $w_1 \Vdash q$, что неверно, значит $w_1 \nVdash p \rightarrow q$, и импликация тривиальна. Следовательно, $w_0 \Vdash (p \rightarrow q) \rightarrow p$, но $w_0 \nVdash p$, поэтому $w_0 \nVdash ((p \rightarrow q) \rightarrow p) \rightarrow p$.

Пример 3. Не всякая контрмодель является цепочкой. Рассмотрим формулу $(p \rightarrow q) \vee (q \rightarrow p)$ и модель с ветвлением:

Здесь $w_0 \nVdash p \rightarrow q$, поскольку $w_1 \Vdash p$, но $w_1 \nVdash q$. Аналогично $w_0 \nVdash q \rightarrow p$, так как $w_2 \Vdash q$, но $w_2 \nVdash p$. Следовательно, $w_0 \nVdash (p \rightarrow q) \vee (q \rightarrow p)$. Эта формула

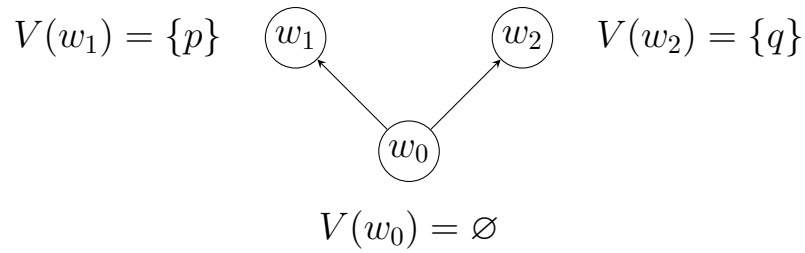


Рисунок 3 — Ветвящаяся модель Крипке

является аксиомой логики Гёделя–Даммита, но не теоремой интуиционистской логики.

Интуиционистская логика играет фундаментальную роль в изоморфизме Карри–Говарда, поскольку именно её конструктивная природа позволяет установить соответствие между логическими доказательствами и программами. В рамках данного соответствия логические формулы интерпретируются как типы, а доказательства — как программы, что делает интуиционистскую логику естественной логической основой типизированных функциональных языков программирования.

1.2 Системы вывода

Формальные системы вывода представляют собой строгие математические конструкции, предназначенные для задания правил построения корректных доказательств в рамках заданной логики. Ниже рассматриваются три основных подхода: аксиоматическая система в стиле Гильберта, натуральный вывод и секвенциальное исчисление.

Система Гильберта

Гильбертова система задаёт логику фиксированным набором схем аксиом и единственным правилом вывода *modus ponens*. Для IPL используются стандартные схемы (A1)–(A9), включающие аксиомы для импликации, конъюнкции, дизъюнкции и *ex falso quodlibet*; полный перечень приведён в приложении Е. Такая формулировка компактна и удобна для металогических рассуждений, но плохо приспособлена для практического построения доказательств: даже тождество $A \rightarrow A$

требует длинной цепочки подстановок в схемы аксиом, а структура рассуждения не выделена явно.

Натуральный вывод

Натуральный вывод [4] представляет собой систему, в которой доказательства строятся с использованием правил введения и устранения для каждой логической связки.

$$\begin{array}{c}
 \frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} \rightarrow I \qquad \frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} \rightarrow E \\
 \\
 \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge I \\
 \\
 \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \wedge E_1 \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} \wedge E_2 \\
 \\
 \frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \vee I_1 \qquad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} \vee I_2 \\
 \\
 \frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C} \vee E \\
 \\
 \frac{\Gamma \vdash \perp}{\Gamma \vdash A} \perp E \qquad \overline{\Gamma, A \vdash A}^{Ax}
 \end{array}$$

Рисунок 4 — Основные правила натурального вывода

Пример. Формула $A \rightarrow (B \rightarrow A)$ выводится так:

$$\frac{\overline{A, B \vdash A}^{Ax}}{\frac{A \vdash B \rightarrow A}{\vdash A \rightarrow (B \rightarrow A)} \rightarrow I} \rightarrow I$$

Рисунок 5 — Вывод формулы $A \rightarrow (B \rightarrow A)$ в натуральном выводе

По сравнению с гильбертовой аксиоматикой натуральный вывод гораздо проще для восприятия человеком: правила введения и устранения связок напрямую отражают ход рассуждения. Вместе с тем такой формат хуже подходит для автоматической обработки: дерево доказательства с произвольным порядком введения гипотез и вложенными подвыводами не столь удобно разбирать алгоритмически, как линейные или почти линейные объекты.

Секвенциальное исчисление

Секвенциальное исчисление [4] основано на понятии секвенции. Пусть $\mathcal{L}$ — множество формул пропозициональной логики, построенных из пропозициональных переменных, связок $\wedge, \vee, \rightarrow$ и константы $\perp$. Секвенцией называется пара (Γ, Δ) , записываемая

$$\Gamma \vdash \Delta,$$

где Γ (антецедент) и Δ (сукцедент) — конечные мультимножества формул из $\mathcal{L}$. Неформально секвенция $\Gamma \vdash \Delta$ читается как «из конъюнкции всех формул Γ выводима дизъюнкция формул Δ ». Секвенция с пустым антецедентом ($\vdash \Delta$) утверждает безусловную выводимость; секвенция с пустым сукцедентом ($\Gamma \vdash$) означает противоречивость Γ .

Ниже приведена система правил классического секвенциального исчисления высказываний (исчисление LK в стиле Генцена).

Секвенция $\vdash A \rightarrow A$ выводится без сечения и структурных правил:

Интуиционистская версия секвенциального исчисления получается ограничением на форму секвенций: интуиционистскими называются те секвенции, в которых справа находится не более одной формулы (то есть выводы строятся в подсистеме, где Δ либо пусто, либо состоит ровно из одной формулы). Ниже приведены правила интуиционистского секвенциального исчисления LJ.

Ключевое отличие от LK — отсутствие правых структурных правил (ослабления, сокращения и перестановки справа), поскольку в сукцеденте находится не более одной формулы. Кроме того, в двухпосылочных правилах ($\rightarrow L, cut$) сукцеденты посылок не объединяются: одна из посылок обязана вывести ровно одну «вспомогательную» формулу, а другая — передать результат в единственную формулу заключения.

Секвенциальный формат удобен для реализации на ЭВМ: правила локальны, вывод представляется деревом с предсказуемой структурой, что облегчает поиск вывода и проверку корректности. Именно на секвенциальных исчислениях и их модификациях (ориентированные варианты, ограничения на правила, стратегии

Аксиомы

$$\overline{A \vdash A}^{Ax} \quad \overline{\Gamma, \perp \vdash \Delta}^{\perp L}$$

Структурные правила

$$\begin{array}{cc} \frac{\Gamma \vdash \Delta}{\Gamma, A \vdash \Delta}^{LW} & \frac{\Gamma \vdash \Delta}{\Gamma \vdash \Delta, A}^{RW} \\ \frac{\Gamma, A, A \vdash \Delta}{\Gamma, A \vdash \Delta}^{LC} & \frac{\Gamma \vdash \Delta, A, A}{\Gamma \vdash \Delta, A}^{RC} \\ \frac{\Gamma, A, B, \Pi \vdash \Delta}{\Gamma, B, A, \Pi \vdash \Delta}^{LX} & \frac{\Gamma \vdash \Delta, A, B, \Pi}{\Gamma \vdash \Delta, B, A, \Pi}^{RX} \end{array}$$

Логические правила

$$\begin{array}{cc} \frac{\Gamma \vdash A, \Phi \quad \Pi, B \vdash \Psi}{\Gamma, \Pi, A \rightarrow B \vdash \Phi, \Psi} \rightarrow L & \frac{\Gamma, A \vdash B, \Delta}{\Gamma \vdash A \rightarrow B, \Delta} \rightarrow R \\ \frac{\Gamma \vdash A, \Delta \quad \Gamma \vdash B, \Delta}{\Gamma \vdash A \wedge B, \Delta} \wedge R \\ \frac{\Gamma, A \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta} \wedge L_1 & \frac{\Gamma, B \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta} \wedge L_2 \\ \frac{\Gamma, A \vdash \Delta \quad \Gamma, B \vdash \Delta}{\Gamma, A \vee B \vdash \Delta} \vee L \\ \frac{\Gamma \vdash A, \Delta}{\Gamma \vdash A \vee B, \Delta} \vee R_1 & \frac{\Gamma \vdash B, \Delta}{\Gamma \vdash A \vee B, \Delta} \vee R_2 \end{array}$$

Правило сечения

$$\frac{\Gamma \vdash A, \Delta \quad \Gamma, A \vdash \Sigma}{\Gamma \vdash \Delta, \Sigma}^{cut}$$

Рисунок 6 — Правила классического секвенциального исчисления LK

$$\frac{\overline{A \vdash A}^{Ax}}{\vdash A \rightarrow A} \rightarrow R$$

Рисунок 7 — Вывод секвенции $\vdash A \rightarrow A$ в LK

устранения сечения и т. д.) строятся многие современные автоматические средства; они служат основой для развитых систем в духе LJT , LJT_{SAT} и аналогов.

Аксиомы

$$\frac{}{A \vdash A} \text{Ax} \quad \frac{}{\Gamma, \perp \vdash \Delta} \perp L$$

Структурные правила

$$\frac{\Gamma \vdash \Delta}{\Gamma, A \vdash \Delta} \text{LW} \quad \frac{\Gamma, A, A \vdash \Delta}{\Gamma, A \vdash \Delta} \text{LC} \quad \frac{\Gamma, A, B, \Gamma' \vdash \Delta}{\Gamma, B, A, \Gamma' \vdash \Delta} \text{LX}$$

Логические правила

$$\frac{\Gamma \vdash A \quad \Gamma, B \vdash \Delta}{\Gamma, A \rightarrow B \vdash \Delta} \rightarrow L \quad \frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} \rightarrow R$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge R$$

$$\frac{\Gamma, A \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta} \wedge L_1 \quad \frac{\Gamma, B \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta} \wedge L_2$$

$$\frac{\Gamma, A \vdash \Delta \quad \Gamma, B \vdash \Delta}{\Gamma, A \vee B \vdash \Delta} \vee L$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \vee R_1 \quad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} \vee R_2$$

Правило сечения

$$\frac{\Gamma \vdash A \quad \Gamma, A \vdash \Delta}{\Gamma \vdash \Delta} \text{cut}$$

Рисунок 8 — Правила интуиционистского секвенциального исчисления LJ

1.3 Типизированное λ -исчисление

В теории доказательств основной задачей является поиск доказательства: его структура, сравнение различных выводов одной и той же формулы. Для конструктивной (интуиционистской) логики это особенно существенно, поскольку доказательство трактуется как конструкция в смысле интерпретации интуиционистской логики.

Удобно явно указывать терм (объект доказательства) рядом с формулой: запись $M : \psi$ означает, что M — доказательство ψ , при контексте гипотез Γ используют суждение $\Gamma \vdash M : \psi$.

Правило устранения импликации в натуральном выводе с аннотациями: если $\Gamma \vdash M : A \rightarrow B$ и $\Gamma \vdash N : A$, то доказательством B задаётся применение $M N$:

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B} \rightarrow_E$$

Рисунок 9 — Правило применения для импликации

Аксиома гипотезы: переменная, помеченная формулой A , служит доказательством A в расширенном контексте:

$$\overline{\Gamma, x : A \vdash x : A}^{Ax}$$

Рисунок 10 — Аксиома гипотезы

Введение импликации: доказательство $A \rightarrow B$ из подвывода с гипотезой $x : A$ записывают как λ -абстракцию по x :

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} \rightarrow_I$$

Рисунок 11 — Правило абстракции для импликации

Таким образом аннотированный натуральный вывод совпадает по виду с правилами типизации в λ -исчислении. В интерпретации интуиционистской логики конструкция для импликации $A \rightarrow B$ — это функция, сопоставляющая конструкции для A конструкции для B .

Аннотирование правил LJ (импликативный фрагмент)

Подход с аннотированным термами к формулам можно распространить и на секвенциальное исчисление:

$$\overline{\Gamma, x : A \vdash x : A}^{Ax} \quad \frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} \rightarrow_R$$

Рисунок 12 — Аннотированные правила Ax и $(\rightarrow R)$

$$\frac{\Gamma \vdash M : A \quad \Delta, z : B \vdash N : C}{\Gamma, \Delta, u : A \rightarrow B \vdash N[z := u M] : C} \rightarrow_L$$

Рисунок 13 — Аннотированное правило $(\rightarrow L)$

Основная суть аннотирования в том, чтобы устранять свободные переменные у терма справа. Под свободными переменными понимаются только те свободные относительно терма справа, которых нет слева. Проще всего объяснить данную идею на правиле $(\rightarrow L)$. В нем мы в результате справа получаем C , но среди посылок у нас пропадает B . Из этого следует, что нужно заменить все вхождения терма z в терме N , на те, что остались слева. Также стоит обосновать почему мы можем использовать терм M при замене. Тут мы знаем, что все свободные переменные терма M содержатся в Γ (левая посылка $\Gamma \vdash M : A$), поэтому подстановка не вносит новых свободных переменных.

В правиле $(\rightarrow R)$ мы убираем из левой части $x : A$, поэтому в правой должны как то убрать свободную переменную x в терме M . Для этого мы просто делаем ее связанной при помощи λ -абстракции.

Суперскрипты для произведения и суммы

Аннотировать можно также и остальные правила секвенциального исчисления, но для этого требуется ввести лямбда-термы для произведения и суммы типов. Для произведения подойдет терм пары следующего вида:

$$\begin{aligned}\langle M, N \rangle &= \lambda x. x M N \\ \pi_1 &= \lambda p. p(\lambda x y. x) \\ \pi_2 &= \lambda p. p(\lambda x y. y)\end{aligned}$$

Рисунок 14 — Термы для произведения типов

Здесь π_1 и π_2 — проекции на первую и вторую компоненту пары. Причем $\pi_1 \langle M, N \rangle \rightarrow_\beta M$ и $\pi_2 \langle M, N \rangle \rightarrow_\beta N$.

Для суммы типов можно использовать следующие термы:

$$\begin{aligned}L &= \lambda e c_1 c_2. c_1 e \\ R &= \lambda e c_1 c_2. c_2 e \\ \text{case } M \text{ of } [x]P \text{ or } [y]Q &= M(\lambda x. P)(\lambda y. Q)\end{aligned}$$

Рисунок 15 — Термы для суммы типов

Тут L и R — конструкторы суммы, а $case$ — элиминатор. В выражении $case$ M может быть вида LV либо RV , где V какой либо терм. Таким образом

$$\begin{aligned} case\ LM\ of\ [x]P\ or\ [y]Q &\rightarrow_{\beta} P[x := M] \\ case\ RM\ of\ [x]P\ or\ [y]Q &\rightarrow_{\beta} Q[y := M] \end{aligned}$$

Рисунок 16 — Редукция разбора случаев для суммы

Аннотирование правил LJ (полный фрагмент)

Теперь благодаря введенным термам для произведения и суммы типов, можно аннотировать остальные правила секвенциального исчисления. Для простоты будем аннотировать только логические правила, так как в структурные аннотируются естественно.

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \langle a, b \rangle : A \wedge B} \wedge R$$

Рисунок 17 — Аннотированное правило $(\wedge R)$

$$\frac{\Gamma, a : A \vdash d : C}{\Gamma, p : A \wedge B \vdash d[a := \pi_1 p] : C} \wedge L_1 \quad \frac{\Gamma, b : B \vdash d : C}{\Gamma, p : A \wedge B \vdash d[b := \pi_2 p] : C} \wedge L_2$$

Рисунок 18 — Аннотированные правила $(\wedge L_1)$ и $(\wedge L_2)$

$$\frac{\Gamma, a : A \vdash d_1 : C \quad \Gamma, b : B \vdash d_2 : C}{\Gamma, p : A \vee B \vdash case\ p\ of\ [a]d_1\ or\ [b]d_2 : C} \vee L$$

Рисунок 19 — Аннотированное правило $(\vee L)$

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash La : A \vee B} \vee R_1 \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash Rb : A \vee B} \vee R_2$$

Рисунок 20 — Аннотированные правила $(\vee R_1)$ и $(\vee R_2)$

2 Конструкторская часть

2.1 Исчисление LJТ

Исчисление LJ в формулировке Дайкхоффа

Секвенция — пара $\Gamma \vdash G$, где Γ — мультимножество формул (антецедент), G — формула (сукцедент). Поскольку антецедент является мультимножеством, перестановка формул допускается автоматически, и структурное правило перестановки не требуется. Формулы строятся из пропозициональных переменных, связок $\wedge, \vee, \rightarrow$ и константы $\perp$. Константа $\perp$ не считается атомом.

Правила исчисления LJ (в формулировке, следующей Дайкхоффу) приведены ниже. Выделенная формула в заключении каждого правила называется главной (principal).

Аксиома и абсурд

$$\frac{}{\underline{A}, \Gamma \vdash A} \text{Ax}, A \text{ — атом} \quad \frac{}{\underline{\perp}, \Gamma \vdash G} \perp L$$

Конъюнкция

$$\frac{A, B, \Gamma \vdash G}{\underline{A \wedge B}, \Gamma \vdash G} \wedge L \quad \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash \underline{A \wedge B}} \wedge R$$

Дизъюнкция

$$\frac{A, \Gamma \vdash G \quad B, \Gamma \vdash G}{\underline{A \vee B}, \Gamma \vdash G} \vee L \quad \frac{\Gamma \vdash A_i}{\Gamma \vdash \underline{A_1 \vee A_2}} \vee R$$

Импликация

$$\frac{\underline{A \rightarrow B}, \Gamma \vdash A \quad B, \Gamma \vdash G}{\underline{A \rightarrow B}, \Gamma \vdash G} \rightarrow L \quad \frac{A, \Gamma \vdash B}{\Gamma \vdash \underline{A \rightarrow B}} \rightarrow R$$

Рисунок 21 — Правила LJ в формулировке Дайкхоффа

Ключевая особенность данной формулировки — правило $(\rightarrow L)$: главная формула $A \rightarrow B$ сохраняется в левой посылке. Тем самым правило сокращения встроено в $(\rightarrow L)$, и ни сокращение, ни ослабление, ни перестановка не требуются как примитивные правила: все три допустимы.

Проблема завершаемости

Для формализации завершаемости вводится вес формулы $\text{wt}(A)$: Форму-

$$\begin{aligned}\text{wt}(p) &= \text{wt}(\perp) = 1, \\ \text{wt}(A \wedge B) &= \text{wt}(A) + \text{wt}(B) + 2, \\ \text{wt}(A \vee B) &= \text{wt}(A) + \text{wt}(B) + 1, \\ \text{wt}(A \rightarrow B) &= \text{wt}(A) + \text{wt}(B) + 1.\end{aligned}$$

Рисунок 22 — Вес формул в доказательстве завершаемости LJТ

лы антецедента и сукцедент секвенции образуют мультимножество; на мультимножествах определяется отношение $\gg$ (мультимножественное упорядочение Дершовица–Манна), которое фундировано: если из мультимножества удалить один или несколько элементов и заменить их произвольным числом элементов меньшего веса, результат строго меньше исходного.

Во всех правилах LJ, кроме $(\rightarrow L)$, каждая посылка строго меньше заключения в смысле $\gg$: главная формула заменяется подформулами меньшего веса. Однако в правиле $(\rightarrow L)$ левая посылка $A \rightarrow B, \Gamma \vdash A$ содержит ту же формулу $A \rightarrow B$, что и заключение, поэтому $\gg$ -убывание не гарантировано. Как следствие, обратный поиск вывода от корня к листьям (root-first proof search) может заикнуться: применение $(\rightarrow L)$ к секвенции $(D_1 \rightarrow D_2) \rightarrow C, \Gamma \vdash G$ порождает подцель $(D_1 \rightarrow D_2) \rightarrow C, \Gamma \vdash D_1 \rightarrow D_2$, в которой главная формула по-прежнему присутствует и может быть использована повторно.

Правила исчисления LJТ

В 1992 году Дайкхофф предложил исчисление LJТ (также известное как G4ip), в котором правило $(\rightarrow L)$ заменяется четырьмя правилами, определяемыми структурой левой части (антецедента) главной импликации $D \rightarrow C$. Все остальные правила сохраняются без изменений.

Случай 1. $D = p$ — пропозициональная переменная:

Случай 2. $D = C_1 \wedge C_2$:

Случай 3. $D = C_1 \vee C_2$:

$$\frac{B, p, \Gamma \vdash G}{p \rightarrow B, \Gamma \vdash G} \rightarrow L_1$$

Рисунок 23 — Правило $(\rightarrow L_1)$ исчисления LJТ

$$\frac{C_1 \rightarrow (C_2 \rightarrow B), \Gamma \vdash G}{(C_1 \wedge C_2) \rightarrow B, \Gamma \vdash G} \rightarrow L_2$$

Рисунок 24 — Правило $(\rightarrow L_2)$ исчисления LJТ

$$\frac{C_1 \rightarrow B, C_2 \rightarrow B, \Gamma \vdash G}{(C_1 \vee C_2) \rightarrow B, \Gamma \vdash G} \rightarrow L_3$$

Рисунок 25 — Правило $(\rightarrow L_3)$ исчисления LJТ

Случай 4. $D = C_1 \rightarrow C_2$:

$$\frac{C_2 \rightarrow B, \Gamma \vdash C_1 \rightarrow C_2 \quad B, \Gamma \vdash G}{(C_1 \rightarrow C_2) \rightarrow B, \Gamma \vdash G} \rightarrow L_4$$

Рисунок 26 — Правило $(\rightarrow L_4)$ исчисления LJТ

Во всех четырёх правилах главная формула не сохраняется ни в одной из посылок: она заменяется формулами строго меньшего веса. Действительно:

- $(\rightarrow L_1)$: формула $p \rightarrow B$ ($\text{wt} = \text{wt}(B) + 2$) заменяется на B ;
- $(\rightarrow L_2)$: $\text{wt}((C_1 \wedge C_2) \rightarrow B) = \text{wt}(C_1) + \text{wt}(C_2) + \text{wt}(B) + 3$, а $\text{wt}(C_1 \rightarrow (C_2 \rightarrow B)) = \text{wt}(C_1) + \text{wt}(C_2) + \text{wt}(B) + 2$;
- $(\rightarrow L_3)$: одна формула веса $\text{wt}(C_1) + \text{wt}(C_2) + \text{wt}(B) + 2$ заменяется двумя формулами весов $\text{wt}(C_1) + \text{wt}(B) + 1$ и $\text{wt}(C_2) + \text{wt}(B) + 1$, каждая строго меньше;
- $(\rightarrow L_4)$: формула $(C_1 \rightarrow C_2) \rightarrow B$ заменяется на $C_2 \rightarrow B$ (в левой посылке) и B (в правой), обе строго меньше.

Таким образом, каждая посылка каждого правила LJТ строго меньше заключения в мультимножественном упорядочении $\gg$, что гарантирует завершаемость обратного поиска вывода. Правила ослабления и сокращения допустимы, но не входят в систему как примитивные.

Обратный поиск вывода

Обратный поиск вывода (root-first proof search) в LJТ — процедура, которая по данной секвенции $\Gamma \vdash G$ определяет, выводима ли она. Процедура сопоставляет секвенцию с заключением одного из правил и рекурсивно доказывает посылки. Все правила LJТ, кроме $(\vee R)$ и $(\rightarrow L_4)$, обратимы: из доказуемости заключения следует доказуемость каждой посылки. При применении обратимого правила возврат (backtracking) не требуется: если хотя бы одна посылка недоказуема, заключение также недоказуемо. Правило $(\rightarrow L_4)$ является правообратимым: из доказуемости заключения следует доказуемость правой посылки $\Gamma, C \vdash G$, тогда как левая посылка может быть недоказуема. Поэтому $(\rightarrow L_4)$ может рассматриваться как правило с одной посылкой и побочным условием (side-condition), а вызов для правой посылки является хвостовой рекурсией.

Возврат необходим только при применении $(\vee R)$ (выбор дизъюнкта) и $(\rightarrow L_4)$ (выбор импликационной клаузы из антецедента).

Псевдокод процедуры приведён в приложении А (листинг 48). Поскольку множество формул в каждой посылке строго меньше в мультимножественном упорядочении $\gg$, рекурсия завершается.

Пример вывода

Покажем вывод формулы $((a \vee (a \rightarrow b)) \rightarrow b) \rightarrow b$ в системе LJТ с λ -термовыми аннотациями.

$$\begin{array}{c}
 \frac{}{g_1 : a \rightarrow b, h : b \rightarrow b \vdash g_1 : a \rightarrow b} Ax \quad \frac{}{y : b, g_1 : a \rightarrow b \vdash y : b} Ax \\
 \hline
 \frac{}{g_1 : a \rightarrow b, g_2 : (a \rightarrow b) \rightarrow b \vdash g_2 g_1 : b} \rightarrow L_4 \\
 \hline
 \frac{}{f : (a \vee (a \rightarrow b)) \rightarrow b \vdash f(R(\lambda x. f(Lx))) : b} \rightarrow L_3 \\
 \hline
 \vdash \lambda f. f(R(\lambda x. f(Lx))) : ((a \vee (a \rightarrow b)) \rightarrow b) \rightarrow b \rightarrow R
 \end{array}$$

Рисунок 27 — Пример LJТ-вывода с λ -термовыми аннотациями

Более сложный пример — вывод формулы $(((((a \wedge b) \rightarrow c) \rightarrow ((a \rightarrow c) \vee (b \rightarrow c)))) \rightarrow c) \rightarrow c$ с аннотированием термами — приведён в приложении Б (с. 74).

2.2 Исчисление LJT_{SAT}

Исчисление LJТ предоставляет завершающуюся процедуру поиска вывода, однако не даёт свидетельства недоказуемости: при неудаче поиска остаётся лишь факт отсутствия вывода, но не контрмодель. Клэссен и Розен [5] предложили инструмент *intuit* — доказатель для интуиционистской пропозициональной логики, построенный поверх инкрементального SAT-решателя по схеме SMT (Satisfiability Modulo Theories). Фиорентини, Горе и Грэм-Ленгран [6] дали теоретико-доказательственное обоснование этого подхода, сформулировав исчисление LJT_{SAT} , которое объединяет поиск вывода в LJТ [7] с использованием SAT-решателя и реализует полную процедуру принятия решения с двумя исходами: вывод в LJT_{SAT} либо контрмодель Крипке.

Клаузальная форма

Исчисление LJT_{SAT} работает с секвенциями в клаузальной форме $R, X \Rightarrow q$, где:

- R — множество плоских клауз вида $\bigwedge A_1 \rightarrow \bigvee A_2$, где $A_1, A_2 \subseteq Atm$ — конечные множества атомов;
- X — множество импликационных клауз вида $(a \rightarrow b) \rightarrow c$, где a, b, c — атомы;
- q — атом (целевая формула).

Произвольная формула α интуиционистской логики преобразуется в клаузальную форму процедурой клаузификации [5].

Шаг 1. Формула α приводится к виду $B \rightarrow q$, где q — атом. Если α не имеет такого вида, вводится свежий атом q и формула заменяется на $(\alpha \rightarrow q) \rightarrow q$ (эквивалентность в интуиционистской логике).

Шаг 2. Формула B раскладывается в множества R и X с помощью правил перезаписи вида $A \rightsquigarrow B_1, \dots, B_n$, заменяющих допущение A на эквивалентный набор допущений B_i . Если формула ещё не является импликацией, применяется правило

$$A \rightsquigarrow \top \rightarrow A.$$

Далее применяются структурные правила, упрощающие вид импликации: где a

$$\begin{aligned}
(A \vee B) \rightarrow a &\rightsquigarrow A \rightarrow a, \quad B \rightarrow a, \\
a \rightarrow (A \wedge B) &\rightsquigarrow a \rightarrow A, \quad a \rightarrow B, \\
A \rightarrow (B \rightarrow C) &\rightsquigarrow (A \wedge B) \rightarrow C,
\end{aligned}$$

Рисунок 28 — Структурные правила клаузификации

— атом или константа $\perp$, $\top$. Для введения атомов на нужных позициях используются правила с введением свежих переменных (обозначены строчными буквами справа): Здесь a, b, c — свежие атомы, не встречающиеся в формуле. Каждая под-

$$\begin{aligned}
A \rightarrow (\dots \vee B \vee \dots) &\rightsquigarrow A \rightarrow (\dots \vee b \vee \dots), \quad b \rightarrow B, \\
(\dots \wedge A \wedge \dots) \rightarrow B &\rightsquigarrow (\dots \wedge a \wedge \dots) \rightarrow B, \quad A \rightarrow a, \\
(A \rightarrow B) \rightarrow C &\rightsquigarrow (a \rightarrow B) \rightarrow C, \quad A \rightarrow a, \\
(A \rightarrow B) \rightarrow C &\rightsquigarrow (A \rightarrow b) \rightarrow C, \quad B \rightarrow b, \\
(A \rightarrow B) \rightarrow C &\rightsquigarrow (A \rightarrow B) \rightarrow c, \quad c \rightarrow C.
\end{aligned}$$

Рисунок 29 — Правила клаузификации с введением свежих атомов

формула именуется не более одного раза, поэтому суммарный размер (R, X, q) линеен относительно размера α .

Результатом клаузификации является тройка $clausal(\alpha) = (R, X, q)$, для которой выполняется $\vdash_{ipl} \alpha$ тогда и только тогда, когда $R, X \vdash_{ipl} q$.

В клаузальной форме из правил LJТ применимо только правило $(\rightarrow L_4)$, так как антецеденты импликационных клауз сами являются импликациями. Плоские клаузы не содержат вложенных связок и допускают классическое рассуждение: $R \vdash_{cpl} q$ тогда и только тогда, когда $R \vdash_{ipl} q$, что позволяет делегировать работу с ними SAT-решателю.

Правила исчисления LJT_{SAT}

Исчисление LJT_{SAT} состоит из трёх правил.

Правило (cpl) . Если множество плоских клауз R классически влечёт q (что проверяется SAT-решателем), секвенция выводима:

$$\frac{R \vdash_{cpl} q}{R, X \Rightarrow q}^{cpl}$$

Рисунок 30 — Правило (*cpl*) исчисления LJT_{SAT}

Правило (cut_{ipl}). Интуиционистское сечение: если левая посылка является интуиционистской теоремой и плоская клауза φ позволяет вывести правую посылку, то заключение выводимо:

$$\frac{R_1, X_1 \vdash_{ipl} \varphi \quad \varphi, R_2, X_2 \Rightarrow q}{R_1, R_2, X_1, X_2 \Rightarrow q}^{cut_{ipl}}$$

Рисунок 31 — Правило (cut_{ipl}) исчисления LJT_{SAT}

Правило (ljt). Обобщённое правило левого введения импликации, соответствующее ($\rightarrow L_4$) из LJТ, адаптированное к клаузальной форме. Пусть $\iota = (a \rightarrow b) \rightarrow c \in X$, $X_\iota = X \setminus \{\iota\}$ и A_1 — множество атомов. Тогда:

$$\frac{R_1, b \rightarrow c, X_\iota, A_1 \Rightarrow b \quad R_2, \bigwedge(A_1 \setminus \{a\}) \rightarrow c, X, \iota \Rightarrow q}{R_1, R_2, X, \iota \Rightarrow q}^{ljt}$$

Рисунок 32 — Правило (ljt) исчисления LJT_{SAT}

Правило (ljt) обобщает ($\rightarrow L_4$) из LJТ в двух направлениях. Во-первых, допускается расщепление контекста (R_1 и R_2 могут различаться). Во-вторых, в левую посылку могут быть добавлены дополнительные атомарные допущения A_1 , что расширяет область применимости правила; ценой этого расширения является ослабление выученной клаузы в правой посылке: вместо атома c появляется плоская клауза $\bigwedge(A_1 \setminus \{a\}) \rightarrow c$.

При $R_1 = R_2 = R$ и $A_1 = \{a\}$ (то есть $\bigwedge(A_1 \setminus \{a\}) \rightarrow c$ вырождается в атом c) правило (ljt) сводится к ($\rightarrow L_4$) исчисления LJТ, поскольку наличие c в правой посылке делает импликационную клаузу ι избыточной.

Техника CEGAR

Техника CEGAR (Counter-Example Guided Abstraction Refinement), предложенная Кларком и соавторами [8], объединяет решение задачи на абстракции с итеративным уточнением по контрпримерам. Общая схема состоит из трёх шагов:

1. Абстракция — исходная задача заменяется более грубой, но проще решаемой;
2. Проверка — по найденному решению или контрпримеру абстракции проверяется, переносится ли результат на исходную задачу;
3. Уточнение — если абстрактное решение ложно, из контрпримера строится новое ограничение, исключающее эту ошибку, после чего цикл повторяется.

В LJT_{SAT} роль абстракции играет классическое рассуждение над плоскими клаузами. SAT-решатель проверяет, выводим ли целевой атом q из R при дополнительных атомарных допущениях A . Классическая логика сильнее интуиционистской для таких подзадач, поэтому SAT может находить «оптимистичный» вывод, недостижимый в IPL. Контрпримером служит классическая модель M — множество истинных атомов, удовлетворяющее $R \cup A$, но не содержащее q : она показывает, что текущая абстракция слишком слаба.

Уточнение выполняется через рекурсивное применение правила (ljt). Для импликационной клаузы, несовместимой с M , либо строится подзадача, либо формируется выученная плоская клауза $\tilde{\varphi}$, добавляемая в SAT-решатель и исключающая модель M из множества классических моделей $\mathcal{M}(R)$. Каждая такая клауза строго уменьшает $\mathcal{M}(R)$, что обеспечивает завершаемость процедуры. Если ни одна ветка не даёт вывода, контрмодель Крипке собирается методом склейки подмоделей.

Тот же принцип CEGAR лежит в основе процедуры `proveR` в исчислении $C^{\rightarrow}$ (подраздел 2.4) и реализован в прототипе через цикл запросов к инкрементальному SAT/SMT-решателю Z3.

Процедура поиска вывода

Процедура поиска вывода в LJT_{SAT} состоит из двух функций: главной `LJTSatMain` и рекурсивной `LJTSat` (псевдокод приведён в Приложении В, листинги 49 и 50). Обе функции используют единый инкрементальный SAT-решатель s : клаузы могут добавляться в s , но не удаляться; запросы принимают переменное множество атомарных допущений.

Функция `LJTSatMain`(R_0, X_0, q_0) инициализирует SAT-решатель s и загружает в него все плоские клаузы из R_0 , а также для каждой импликационной клаузы

$(a \rightarrow b) \rightarrow c \in X_0$ — клаузу $b \rightarrow c$ (следствие исходной задачи). После этого управление передаётся функции LJTSat.

Функция $\text{LJTSat}(R, X, A, q)$ реализует описанную в подразделе 2.2 схему CEGAR. Она запрашивает SAT-решатель: выполняется ли $R(s), A \vdash_{cpl} q$? Если да, секвенция выводима по правилу (*cpl*); SAT-решатель возвращает подмножество $A' \subseteq A$, достаточное для вывода. В противном случае SAT-решатель возвращает классическую контрмодель M — множество атомов, удовлетворяющее $R(s) \cup A$, но не q .

Контрмодель M проверяется на совместимость с импликационными клаузами. Для каждой $\iota = (a \rightarrow b) \rightarrow c \in X$, такой что $a \notin M, b \notin M$ и $c \notin M$ (то есть M не удовлетворяет ι тривиально), выполняется рекурсивный вызов для левой посылки правила (*ljt*): $\text{LJTSat}(R \cup \{b \rightarrow c\}, X_\iota, M \cup \{a\}, b)$, где $X_\iota = X \setminus \{\iota\}$. Если вызов вернул контрмодель K_1 , она сохраняется для последующей склейки. Если же вызов успешен и возвращает множество атомов A_1 , из него формируется выученная клауза $\tilde{\varphi} = \bigwedge(A_1 \setminus \{a\}) \rightarrow c$, которая добавляется в SAT-решатель. Затем LJTSat вызывается рекурсивно для правой посылки: $\text{LJTSat}(R \cup \{\tilde{\varphi}\}, X, A, q)$. Этот вызов является хвостовой рекурсией (правообратимость правила (*ljt*)) и соответствует перезапуску цикла CEGAR с уточнённым множеством клауз.

Если для всех применимых импликационных клауз рекурсивные вызовы вернули контрмодели, формула недоказуема, и из M и накопленных подмоделей конструируется контрмодель Крипке методом склейки.

Завершаемость процедуры гарантируется убыванием пары $(X, \mathcal{M}(R))$ по лексикографическому порядку, где $\mathcal{M}(R)$ — множество классических моделей, удовлетворяющих R . Каждая выученная клауза $\tilde{\varphi}$ исключает хотя бы одну модель (а именно M) из $\mathcal{M}(R)$, что гарантирует строгое убывание по второй компоненте при неизменной первой.

Построение контрмодели Крипке

При неудаче поиска вывода процедура строит контрмодель Крипке методом склейки (*gluing*). Каждый неудачный лист дерева поиска задаёт мир контрмодели, а рекурсивная структура вызовов определяет отношение достижимости.

Пусть $\Theta = (K_\iota)_{\iota \in X_M}$ — семейство крипке-моделей, построенных рекурсивными вызовами для импликационных клауз $X_M = \{\iota \in X \mid a \notin M, b \notin$

$M, c \notin M\}$, и M — классическая контрмодель. Тогда модель $\text{Mod}(r, \Theta, M) = \langle W, \preceq, r, V \rangle$ определяется так:

- $W = \{r\} \cup \bigcup_{i \in X_M} W_i$, где W_i — множество миров модели K_i ;
- r — новый корневой мир с оценкой $V(r) = M$;
- $r \preceq r_i$ для каждого корня r_i модели K_i ;
- $\preceq$ — рефлексивно-транзитивное замыкание отношений достижимости всех подмоделей и связей $r \preceq r_i$.

Если $\Theta = \emptyset$ (все импликационные клаузы имеют хотя бы один компонент, истинный в M), контрмодель состоит из единственного мира r с $V(r) = M$. Условие наследуемости выполняется по построению: корни подмоделей r_i удовлетворяют $r_i \models M$, поскольку $M \subseteq V(r_i)$.

Корректность построения гарантируется тем, что в корне r вынуждены все формулы из $R \cup X$, но не вынуждена целевая формула q , что делает построенную структуру контрмоделью для исходной секвенции.

2.3 Аннотирование термами

В исследовательской части было показано, что правила секвенциального исчисления LJ могут быть аннотированы λ -термами в соответствии с изоморфизмом Карри–Говарда: каждому выводу сопоставляется терм, типом которого является выведенная формула. Тот же подход применим к исчислениям LJТ и к правилам клаузификации, что позволяет по найденному выводу извлечь программу (свидетель населённости типа).

Аннотирование правил LJТ

Правила LJТ, общие с LJ (аксиома, $\perp L$, $\wedge L$, $\wedge R$, $\vee L$, $\vee R$, $\rightarrow R$), аннотируются так же, как в LJ. Специфика LJТ состоит в четырёх правилах левого введения импликации.

Правило ($\rightarrow L_1$). Атом p присутствует в антецеденте, поэтому терм для B получается применением f к x :

Правило ($\rightarrow L_2$). Импликация $(A \wedge B) \rightarrow C$ заменяется на $A \rightarrow (B \rightarrow C)$:

$$\frac{\Gamma, x : p, y : B \vdash t : G}{\Gamma, x : p, f : p \rightarrow B \vdash t[y := f x] : G} \rightarrow L_1$$

Рисунок 33 — Аннотированное правило $(\rightarrow L_1)$

$$\frac{\Gamma, g : A \rightarrow (B \rightarrow C) \vdash t : G}{\Gamma, f : (A \wedge B) \rightarrow C \vdash t[g := \lambda x. \lambda y. f \langle x, y \rangle] : G} \rightarrow L_2$$

Рисунок 34 — Аннотированное правило $(\rightarrow L_2)$

Правило $(\rightarrow L_3)$. Импликация $(A \vee B) \rightarrow C$ заменяется парой $A \rightarrow C$ и $B \rightarrow C$:

$$\frac{\Gamma, g_1 : A \rightarrow C, g_2 : B \rightarrow C \vdash t : G}{\Gamma, f : (A \vee B) \rightarrow C \vdash t[g_1 := \lambda x. f(Lx), g_2 := \lambda y. f(Ry)] : G} \rightarrow L_3$$

Рисунок 35 — Аннотированное правило $(\rightarrow L_3)$

Правило $(\rightarrow L_4)$. Единственное правило с двумя посылками. В левой посылке формула $(C_1 \rightarrow C_2) \rightarrow B$ заменяется на $C_2 \rightarrow B$:

$$\frac{\Gamma, g : C_2 \rightarrow B \vdash s : C_1 \rightarrow C_2 \quad \Gamma, y : B \vdash t : G}{\Gamma, f : (C_1 \rightarrow C_2) \rightarrow B \vdash t[y := f s', g := \lambda z. f(Kz)] : G} \rightarrow L_4$$

Рисунок 36 — Аннотированное правило $(\rightarrow L_4)$

где $K = \lambda a. \lambda b. a$ — комбинатор константы, а $s' = s[g := \lambda z. f(Kz)]$. Интуиция подстановки: имея $f : (C_1 \rightarrow C_2) \rightarrow B$ и значение $z : C_2$, можно построить константную функцию $Kz = \lambda x^{C_1}. z : C_1 \rightarrow C_2$ и применить f , получив $f(Kz) : B$. Таким образом $g := \lambda z. f(Kz) : C_2 \rightarrow B$. Развёрнутый пример подстановки приведён в приложении Б (с. 74).

Клаузификация как секвенциальные правила

Каждое правило перезаписи, используемое при клаузификации формулы в клаузальную форму $R, X \Rightarrow q$, может быть представлено как допустимое правило секвенциального исчисления. Это позволяет аннотировать клаузификацию λ -термами: каждому шагу перезаписи соответствует терм, преобразующий доказательство в клаузальной форме в доказательство исходной формулы.

Рассмотрим правила клаузификации. Каждое из них имеет вид: секвенция с допущением $B_1, \dots, B_n$ выводима тогда и только тогда, когда выводима секвенция с допущением A . Аннотирование состоит в указании терма, реализующего это преобразование.

Начальное правило. Формула, не являющаяся импликацией, оборачивается:

$$\frac{\Gamma, f : \top \rightarrow A \vdash t : G}{\Gamma, x : A \vdash t[f := \lambda u. x] : G}$$

Рисунок 37 — Аннотированное начальное правило клаузификации

Дизъюнкция слева от стрелки:

$$\frac{\Gamma, f_1 : A \rightarrow a, f_2 : B \rightarrow a \vdash t : G}{\Gamma, f : (A \vee B) \rightarrow a \vdash t[f_1 := \lambda x. f(Lx), f_2 := \lambda y. f(Ry)] : G}$$

Рисунок 38 — Аннотированное правило клаузификации для дизъюнкции слева

Конъюнкция справа от стрелки:

$$\frac{\Gamma, f_1 : a \rightarrow A, f_2 : a \rightarrow B \vdash t : G}{\Gamma, f : a \rightarrow (A \wedge B) \vdash t[f_1 := \lambda x. \pi_1(fx), f_2 := \lambda x. \pi_2(fx)] : G}$$

Рисунок 39 — Аннотированное правило клаузификации для конъюнкции справа

Каррирование:

$$\frac{\Gamma, g : (A \wedge B) \rightarrow C \vdash t : G}{\Gamma, f : A \rightarrow (B \rightarrow C) \vdash t[g := \lambda p. f(\pi_1 p)(\pi_2 p)] : G}$$

Рисунок 40 — Аннотированное правило клаузификации для каррирования

Правила именования подформул. При клаузификации для сложной подформулы B вводится свежий атом b , играющий роль определяющего сокращения: $b \equiv B$. Поскольку b и B отождествляются на уровне типов, все правила именования имеют тривиальную термовую аннотацию. Рассмотрим два характерных случая.

Именованье в положительной позиции (например, в сукцеденте дизъюнкции). Подформула B заменяется на b , добавляется клауза $b \rightarrow B$:

Именованье в отрицательной позиции (например, в антецеденте дизъюнкции). Подформула B заменяется на b , добавляется клауза $B \rightarrow b$:

$$\frac{\Gamma, f' : A \rightarrow (\dots \vee b \vee \dots), h : b \rightarrow B \vdash t : G}{\Gamma, f : A \rightarrow (\dots \vee B \vee \dots) \vdash t[f' := f, h := \lambda x. x] : G}$$

Рисунок 41 — Аннотированное правило именования в положительной позиции

$$\frac{\Gamma, f' : (\dots \vee b \vee \dots) \rightarrow a, h : B \rightarrow b \vdash t : G}{\Gamma, f : (\dots \vee B \vee \dots) \rightarrow a \vdash t[f' := f, h := \lambda x. x] : G}$$

Рисунок 42 — Аннотированное правило именования в отрицательной позиции

В обоих случаях подстановка тождественна: $f' := f$ и $h := \lambda x. x$, поскольку b является определительным сокращением для B и значения этих типов совпадают. Аналогичная аннотация применяется ко всем остальным позициям именования (конъюнкция, импликация).

Совокупность аннотированных правил клаузификации и аннотированных правил LJТ позволяет по выводу в LJT_{SAT} построить λ -терм, являющийся свидетелем населённости исходного типа.

Извлечение терма из клаузальной формы

При клаузификации формула α приводится к виду $(\alpha \rightarrow q) \rightarrow q$, где q — свежий атом, не входящий в α . Между α и $(\alpha \rightarrow q) \rightarrow q$ имеет место эквивалентность доказуемости: $\vdash_{ipl} \alpha$ тогда и только тогда, когда $\vdash_{ipl} (\alpha \rightarrow q) \rightarrow q$.

Допустим, процедура поиска вывода в LJT_{SAT} завершилась успешно и по аннотированному выводу построен замкнутый λ -терм $M : (\alpha \rightarrow q) \rightarrow q$. Поскольку q — свежий атом, не входящий в α , к доказательству применима равномерная подстановка: замена атома q на произвольную формулу ψ во всём выводе сохраняет его корректность. При этом структура λ -терма M не изменяется — подстановка затрагивает только типовые аннотации, но не сам терм.

Выполним подстановку $q := \alpha$. Терм M приобретает тип $(\alpha \rightarrow \alpha) \rightarrow \alpha$. Тождественная функция $I = \lambda x. x$ имеет тип $\alpha \rightarrow \alpha$, поэтому применение

$$M I \rightarrow_{\beta} N$$

даёт терм $N : \alpha$, являющийся свидетелем населённости типа α .

2.4 Исчисление $C^{\rightarrow}$

Фиорентини [9] предложил упрощённый вариант исчисления LJT_{SAT} , названный $C^{\rightarrow}$, а также процедуру поиска вывода `proveR` (prove with Restart), реализованную в инструменте `intuitR`. По сравнению с LJT_{SAT} исчисление $C^{\rightarrow}$ не содержит правила сечения и правила (ljt) ; вместо них используется единственное правило (cpl_1) , в котором левая посылка проверяется классически. Благодаря этому выводы в $C^{\rightarrow}$ имеют линейную структуру (единственная ветвь), что существенно упрощает как сам вывод, так и извлечение из него λ -терма.

Правила исчисления

Исчисление $C^{\rightarrow}$ работает с r -секвенциями $R, X \Rightarrow g$ (того же вида, что и в LJT_{SAT}) и состоит из двух правил.

Правило (cpl_0) . Аксиома: если плоские клаузы R классически влекут g , секвенция выводима:

$$\frac{R \vdash_{cpl} g}{R, X \Rightarrow g} \text{ } cpl_0$$

Рисунок 43 — Правило (cpl_0) исчисления $C^{\rightarrow}$

Правило (cpl_1) . Пусть $(a \rightarrow b) \rightarrow c \in X$ и $A \subseteq \mathcal{V}$ — множество пропозициональных переменных (локальные допущения). Если $R, A \vdash_{cpl} b$, формируется выученная клауза $\varphi = \bigwedge(A \setminus \{a\}) \rightarrow c$:

$$\frac{R, A \vdash_{cpl} b \quad R \cup \{\varphi\}, X \Rightarrow g}{R, X \Rightarrow g} \text{ } cpl_1$$

Рисунок 44 — Правило (cpl_1) исчисления $C^{\rightarrow}$

Вывод в $C^{\rightarrow}$ представляет собой линейную последовательность применений правила (cpl_1) , завершающуюся применением (cpl_0) : на каждом шаге k множество плоских клауз R_k пополняется выученной клаузой φ_k , и следующий шаг работает с $R_{k+1} = R_k \cup \{\varphi_k\}$.

Корректность правила (cpl_1) основана на следующем свойстве: если $R, A \vdash_{cpl} b$, то $R, (a \rightarrow b) \rightarrow c \vdash_{ipl} \varphi$. Действительно, из $R, A \vdash_{cpl} b$ по лемме о плоских клаузах следует $R, A \vdash_{ipl} b$, откуда $R, A \setminus \{a\} \vdash_{ipl} a \rightarrow b$, и с

учётom импликационной клаузы получаем $R, (a \rightarrow b) \rightarrow c, A \setminus \{a\} \vdash_{ipl} c$, что даёт $R, (a \rightarrow b) \rightarrow c \vdash_{ipl} \varphi$.

Аннотирование правил

Как и для LJТ, г-секвенции $R, X \Rightarrow g$ могут быть аннотированы λ -термами: каждой плоской клаузе из R сопоставляется переменная, импликационной клаузе $(a \rightarrow b) \rightarrow c \in X$ — переменная $h : (a \rightarrow b) \rightarrow c$, заключению — терм $t : g$.

Правило (cpl_0). Если $R \vdash_{cpl} g$, терм $t : g$ строится из переменных плоских клауз R ; множество X в аннотации не участвует:

$$\frac{R \vdash_{cpl} g}{R, X \vdash t : g}^{cpl_0}$$

Рисунок 45 — Аннотированное правило (cpl_0)

Сечение (cut_{ipl}). Правило (cpl_1) реализует сечение с синтезированным доказательством выученной клаузы. Общая схема:

$$\frac{\Gamma \vdash s : \varphi \quad \Delta, f : \varphi \vdash t : g}{\Gamma, \Delta \vdash t[f := s] : g}^{cut_{ipl}}$$

Рисунок 46 — Аннотированная схема сечения

Правило (cpl_1). Пусть $\iota = (a \rightarrow b) \rightarrow c \in X$, $A \subseteq \mathcal{V}$ и $R, A \vdash_{cpl} b$. SAT-решатель возвращает минимальное подмножество $A' \subseteq A$, достаточное для вывода; по лемме о плоских клаузах существует терм $p : b$ в контексте R и переменных $x_q : q$ для $q \in A'$. Выученная клауза $\varphi = \bigwedge (A \setminus \{a\}) \rightarrow c$ в каррированном виде имеет тип $u_1 \rightarrow (\dots \rightarrow c)$ для атомов $u_i \in A \setminus \{a\}$. Терм для φ :

$$s = \lambda \vec{u}. h(\lambda x^a. p'),$$

где $\vec{u}$ — последовательность абстракций по атомам из $A \setminus \{a\}$, а p' — терм p с подстановкой соответствующих $\vec{u}$ вместо переменных из A' . Аннотированное правило:

$$\frac{R, A \vdash_{cpl} b \quad R, f : \varphi, X \vdash t : g}{R, h : (a \rightarrow b) \rightarrow c, X \vdash t[f := s] : g}^{cpl_1}$$

Рисунок 47 — Аннотированное правило (cpl_1)

Левая посылка не содержит λ -терма — только классическое свидетельство A' . Терм s синтезируется из переменной h импликационной клаузы и существования $p : b$. Правило (cpl_1) — линейная замена связки $(ljt) + (cut_{ipl})$: вместо явного IPL-вывода b слева используется классическая проверка, а подстановка $t[f := s]$ выполняет сечение с выученной клаузой φ .

Пример. Пусть $A = \{a\}$, тогда $A \setminus \{a\} = \emptyset$ и $\varphi = c$. Если $R, \{a\} \vdash_{cpl} b$ и $p : b$ при $x : a$, то $s = h(\lambda x^a. p) : c$, и заключение имеет вид $t[f := h(\lambda x^a. p)] : g$.

Процедура поиска вывода

Процедура $\text{proveR}(R, X, g)$ реализует поиск вывода в $C^\rightarrow$ с помощью двух вложенных циклов и инкрементального SAT-решателя s . Как и LJTSat, она следует схеме CEGAR (подраздел 2.2): SAT проверяет классическую выводимость, контрмодель порождает выученную клаузу или ветвление по импликационным клаузам.

Главный цикл начинается с пустого множества интерпретаций W и вызова SAT-решателя: $R(s) \vdash_{cpl} g$? Если ответ положительный, секвенция выводима по правилу (cpl_0) . В противном случае SAT-решатель возвращает классическую контрмодель M , и начинается внутренний цикл.

Внутренний цикл пополняет множество W интерпретациями и ищет пару $\langle w, \lambda \rangle$, где $w \in W$, $\lambda = (a \rightarrow b) \rightarrow c \in X$ и w не реализует λ в модели $K(W)$. Если такой пары нет, $K(W)$ является контрмоделью Крипке и процедура завершается с ответом CountSat. Если пара найдена, SAT-решателю передаётся запрос: $R(s), w \cup \{a\} \vdash_{cpl} b$? При отрицательном ответе возвращённая интерпретация M добавляется в W , и внутренний цикл продолжается. При положительном ответе из возвращённых допущений A формируется выученная клауза $\varphi = \bigwedge(A \setminus \{a\}) \rightarrow c$, которая добавляется в SAT-решатель, множество W очищается и главный цикл перезапускается (restart).

Завершаемость главного цикла гарантируется тем, что выученные клаузы попарно не классически эквивалентны: каждая новая клауза φ_k ложна на интерпретации w_k , а все ранее выученные — истинны на ней. Завершаемость внутреннего цикла следует из строгого роста W на каждой итерации при конечном универсуме $U(s)$.

Преимущества перед LJT_{SAT}

Исчисление $C^{\rightarrow}$ и процедура `proveR` обладают рядом преимуществ по сравнению с LJT_{SAT} и исходной процедурой `intuit` [5].

- **Линейная структура вывода.** Вывод в $C^{\rightarrow}$ представляет собой единственную ветвь без сечений, тогда как вывод в LJT_{SAT} может содержать ветвления (правило (ljt)) и применения правила сечения. Это упрощает извлечение λ -терма.
- **Компактные контрмодели.** Перезапуск (`restart`) очищает множество W , исключая избыточные миры. На бенчмарке SYJ212 ($k = 25$) контрмодель `intuitR` содержит 4 мира против 1955 у `intuit`.
- **Производительность.** На стандартном наборе из 1200 бенчмарков `intuitR` решает больше задач и в целом работает быстрее, чем `intuit`, `fCube` и `intHistGC`.

3 Технологическая часть

В рамках работы разработан программный прототип доказателя формул интуиционистской пропозициональной логики. Прототип реализует описанный выше подход на основе клаузификации, исчисления $C^{\rightarrow}$, инкрементального SAT/SMT-решателя и последующего аннотирования найденного вывода λ -термами. Основная цель реализации состоит не только в ответе на вопрос о выводимости формулы, но и в построении артефактов, полезных для визуализации соответствия Карри–Говарда: дерева доказательства, контрмодели Крипке и терма, свидетельствующего о населённости соответствующего типа.

3.1 Выбранные технологии

Прототип реализован на языке Haskell. Этот выбор обусловлен тем, что структуры формул, секвенций, клауз, правил вывода и λ -термов естественно описываются алгебраическими типами данных. Кроме того, сильная статическая типизация позволяет разделить несколько уровней представления: неаннотированные деревья доказательств, деревья с аннотациями SAT-решателя и деревья с λ -термами.

Сборка проекта выполняется с помощью Stack. В качестве внешнего решателя используется Z3; он применяется как инкрементальный классический SAT/SMT-оракул для проверки плоских клауз и извлечения ядра невыполнимости (unsat core). Основными библиотеками реализации являются:

- `z3` — низкоуровневые привязки к API Z3;
- `containers` — структуры Map и Set для хранения окружений, множеств атомов и миров контрмодели;
- `mtl` — монада состояния для клаузификации и генерации свежих переменных;
- `fmt` — форматирование формул, секвенций и графов DOT.

3.2 Основные структуры данных и классы

Реализация построена вокруг небольшого набора алгебраических типов данных. Каждый такой тип соответствует объекту из теоретической части: формуле,

клаузе, секвенции, правилу вывода, контрмодели или λ -терму. Благодаря этому переходы между фазами алгоритма выражаются как преобразования типизированных структур, а не как обработка строкового представления формул.

`Atom_` описывает атомарные формулы. В нём различаются обычная пропозициональная переменная, константа истины $\top$ и константа ложности $\perp$. Этот тип используется во всех последующих структурах, поскольку после клаузификации левая и правая части клауз состоят только из атомов.

`Formula_` задаёт синтаксическое дерево исходной формулы IPL. В нём есть четыре конструктора: импликация, конъюнкция, дизъюнкция и атом. Импликация, конъюнкция и дизъюнкция хранят списки подформул; такое представление удобно для правил, которые раскладывают цепочки импликаций или многоместные конъюнкции и дизъюнкции.

Класс `Formula` задаёт общий интерфейс для объектов, которые могут рассматриваться как формулы. Он предоставляет операции извлечения атомов, попытки распознать специализированную структуру из общей формулы и обратного преобразования в `Formula_`. За счёт этого одни и те же функции могут работать с формулами, атомами, плоскими клаузами и импликационными клаузами.

Листинг 1: Тип атомарных формул `Atom_`

```
1 data Atom_ = Top | Bottom | Variable String
2   deriving (Eq, Ord, Show)
```

Листинг 1 фиксирует минимальный набор атомов, который затем используется в формулах, клаузах, секвенциях и моделях.

Листинг 2: Тип формул `Formula_`

```
1 data Formula_ =
2   Implication [Formula_] |
3   Conjunction [Formula_] |
4   Disjunction [Formula_] |
5   Atom Atom_
6   deriving (Eq, Ord, Show)
```

Листинг 2 показывает представление синтаксического дерева исходной формулы до перехода к клаузальной форме.

Листинг 3: Класс преобразования в формулу Formula

```
1 class Formula f where
2   atoms :: f -> [Atom_]
3   fromFormula :: Formula_ -> Maybe f
4   toFormula :: f -> Formula_
```

Листинг 3 задаёт интерфейс, через который специализированные структуры распознаются в общей формуле и собираются обратно.

`Annotated_ a b` — универсальная обёртка для хранения аннотаций. Значение типа `b` содержит сам математический объект, а значение типа `a` содержит дополнительную информацию, зависящую от фазы алгоритма. На ранних этапах аннотация может быть пустой; при работе с Z3 в ней хранится AST-узел решателя; после аннотирования доказательства — λ -терм.

Листинг 4: Универсальная аннотация Annotated_

```
1 data Annotated_ a b = Annotated {
2   annotation :: a,
3   annotated :: b
4 }
```

В листинге 4 показано, что аннотация хранится рядом с объектом и может меняться независимо от логического содержимого.

`Flat_` представляет плоскую клаузу вида $\bigwedge A_i \rightarrow \bigvee B_j$. В реализации она хранит два списка аннотированных атомов: список посылок и список заключений. Такие клаузы добавляются в Z3 и образуют классическую часть задачи.

`Impl_` представляет импликационную клаузу вида $(a \rightarrow b) \rightarrow c$. Именно такие клаузы образуют множество X в исчислении $C^{\rightarrow}$ и выбираются во внутреннем CEGAR-цикле, когда построенная модель нарушает интуиционистскую импликацию.

`Unclausified_` описывает промежуточное состояние клаузификации. Оно содержит зоны R, X, U и цель: уже полученные плоские клаузы, импликационные клаузы, ещё не обработанные формулы и целевой атом. Эта структура нужна для пошагового применения правил клаузификации.

Intuit_ описывает r -секвенцию исчисления $C^{\rightarrow}$: множество плоских клауз R , множество импликационных клауз X и цель. Она получается после завершения клаузации и является входом для основного CEGAR-поиска.

Classic_ описывает классическую подзадачу вида $R, A \vdash g$, где R — плоские клаузы, A — локальные атомарные допущения, а g — цель. Такие секвенции появляются в правилах CPL0 и CPL1, а затем доказываются LJТ-процедурой.

Листинг 5: Тип плоской клаузы Flat_

```
1 data Flat_ a =
2   Flat [Annotated_ a Atom_] [Annotated_ a Atom_]
```

Листинг 5 отражает форму плоской клаузы как пары списков атомов: посылок и возможных заключений.

Листинг 6: Тип импликационной клаузы Impl_

```
1 data Impl_ a =
2   Impl (Annotated_ a Atom_)
3         (Annotated_ a Atom_)
4         (Annotated_ a Atom_)
```

Листинг 6 показывает компактное представление клаузы $(a \rightarrow b) \rightarrow c$ тремя атомами.

Листинг 7: Секвенция фазы клаузации Unclassified_

```
1 data Unclassified_ a c =
2   Unclassified
3     [Annotated_ a (Flat_ c)]
4     [Annotated_ a (Impl_ c)]
5     [Annotated_ a Formula_]
6     (Annotated_ (a, c) Atom_)
```

В листинге 7 выделена зона U , где хранятся формулы, ещё не переведённые в плоские или импликационные клаузы.

Листинг 8 соответствует r -секвенции $R, X \Rightarrow g$, которая является входом алгоритма $C^{\rightarrow}$.

Листинг 8: r-секвенция Intuit_

```
1 data Intuit_ a c =  
2   Intuit  
3     [Annotated_ a (Flat_ c)]  
4     [Annotated_ a (Impl_ c)]  
5     (Annotated_ (a, c) Atom_)
```

Листинг 9: Классическая секвенция Classic_

```
1 data Classic_ a c =  
2   Classic  
3     [Annotated_ a (Flat_ c)]  
4     [Annotated_ (a, c) Atom_]  
5     (Annotated_ (a, c) Atom_)
```

Листинг 9 показывает форму классической подзадачи: плоские клаузы, локальные атомарные допущения и атомарная цель.

ClausificationRule_, CArrowRule_ и LJTRule_ описывают не только факт успешного применения правила, но и дерево вывода. Благодаря этому история работы алгоритма сохраняется в структурированном виде и может быть затем аннотирована λ -термами или выведена как граф.

В листинге 10 каждый конструктор соответствует одному правилу преобразования формул в клаузальную форму. Список Substitution_ хранит термовые подстановки, которые появляются при последующем аннотировании доказательства.

Листинг 11 показывает как исходные правила CPL0/CPL1, так и их расширенные версии с LJT-поддеревьями и подстановками, возникающими при аннотировании.

Solver_ инкапсулирует состояние Z3: контекст, инкрементальный решатель и соответствие между атомами задачи и булевыми AST-узлами. Через эту структуру выполняются две операции: добавление плоских клауз и проверка классической выводимости цели при заданных атомарных допущениях.

CounterModel_ и World представляют конечную контрмодель Крипке. Мир хранит множество истинных в нём атомов, а CounterModel_ хранит сами миры

Листинг 10: Дерево правил клаузации ClausificationRule_

```

1 data ClausificationRule_ a c =
2   AsImpl (Unclassified_ a c) (ClausificationRule_ a c) Formula_ (Impl_ c)
   [Substitution_] |
3   AsFlat (Unclassified_ a c) (ClausificationRule_ a c) Formula_ (Flat_ c)
   [Substitution_] |
4   ImplImpliesFlat (Unclassified_ a c) (ClausificationRule_ a c) (Impl_ c)
   (Flat_ c) [Substitution_] |
5   MakeImpl (Unclassified_ a c) (ClausificationRule_ a c) Formula_
   Formula_ [Substitution_] |
6   LeftDs (Unclassified_ a c) (ClausificationRule_ a c) Formula_
   [Formula_] [Substitution_] |
7   RightCs (Unclassified_ a c) (ClausificationRule_ a c) Formula_
   [Formula_] [Substitution_] |
8   Uncurry (Unclassified_ a c) (ClausificationRule_ a c) Formula_ Formula_
   [Substitution_] |
9   Aliasing (Unclassified_ a c) (ClausificationRule_ a c) Formula_
   Formula_ [Formula_] [Substitution_] |
10  FinishClausification (Unclassified_ a c) (Intuit_ a c) |
11  StartCArrow (Unclassified_ a c) (CArrowRule_ a c)

```

Листинг 11: Дерево правил $C \rightarrow$ CArrowRule_

```

1 data CArrowRule_ a c =
2   CPL0 (Intuit_ a c) (Classic_ a c) |
3   CPL1 (Intuit_ a c) (Classic_ a c) (CArrowRule_ a c) (Flat_ c) (Impl_ c)
   |
4   ExCPL0 (Intuit_ a c) (LJTRule_ a c) [Substitution_] |
5   ExCPL1 (Intuit_ a c) (LJTRule_ a c) (CArrowRule_ a c) (Flat_ c) (Impl_
   c) [Substitution_]

```

и отношение достижимости между ними. Эта структура строится в случае, если CEGAR-процедура не может получить новую выученную клаузу и тем самым обнаруживает невалидность формулы.

Листинг 12: Состояние решателя Solver_

```

1 data Solver_ =
2   Solver
3     Z3.Context
4     Z3.Solver
5     [Annotated_ Z3.AST Atom_]

```


Листинг 12 показывает состояние, необходимое для инкрементального взаимодействия с Z3.

Листинг 13: Результат SAT-проверки SatProveResult_

```
1 data SatProveResult_ =  
2   Yes [Annotated_ Z3.AST Atom_] |  
3   No  [Annotated_ Z3.AST Atom_]
```

В листинге 13 результат SAT-запроса разделён на успешное классическое доказательство и контрпример-модель.

Листинг 14: Мир контрмодели World

```
1 data World a =  
2   World Int (Set (Annotated_ a Atom_))
```

Листинг 14 задаёт один мир контрмодели как номер и множество атомов, истинных в этом мире.

Листинг 15: Контрмодель Крипке CounterModel_

```
1 data CounterModel_ a = CounterModel {  
2   reachableTo :: Map Int [Int],  
3   reachableFrom :: Map Int [Int],  
4   worlds :: Map Int (World a)  
5 }
```

Листинг 15 показывает, что контрмодель хранит и сами миры, и отношение достижимости между ними.

Lambda_ описывает термы расширенного λ -исчисления: переменные, абстракции, применения, пары, проекции, вложения в сумму, разбор случаев и вспомогательные конструкции для подстановок. Этот тип используется на последней фазе, когда из дерева доказательства извлекается программное содержание.

Листинг 16 перечисляет термовые конструкции, соответствующие импликации, конъюнкции и дизъюнкции по Карри–Говарду. Некоторые конструкции являются специальными термами, которые используются для компактной записи операций над произведениями и суммами типов:

Листинг 16: Термы расширенного λ -исчисления

```
1 data Lambda_ =  
2   Abstraction [String] Lambda_ |  
3   Const Lambda_ |  
4   Variable { varName :: String } |  
5   Case Lambda_ [(String, Lambda_)] |  
6   Sum Int Int Lambda_ |  
7   Product [Lambda_] |  
8   Proj Int Int Lambda_ |  
9   Insert Int Int Lambda_ Lambda_ |  
10  Application [Lambda_]
```

- `Const` печатается как `K`. Это абстракция, которая принимает одну переменную и возвращает зафиксированное тело;
- `Case` печатается в виде `Case(expr, [c]Body...)`. Здесь `expr` имеет тип суммы. Если `expr = Sum(n, i, e)`, где `n` — размер суммы типов, `i` — номер выбранного варианта, а `e` — значение выбранного типа, то результатом редукции является `Body_i[c := e]`;
- `Product` задаёт терм произведения типов своих операндов;
- `Proj(n, i, p)` задаёт i -ю проекцию пары `p` длины `n`;
- `Insert(n, i, p, e)` задаёт вставку элемента `e` в пару `p` длины `n` на позицию `i`.

Листинг 17: Подстановка `Substitution_`

```
1 data Substitution_ =  
2   Substitution String Lambda_
```

Листинг 17 задаёт одну операцию замены: имя переменной и терм, который должен быть подставлен вместо неё.

Для большинства перечисленных структур определены экземпляры стандартных классов `Functor` и `Bifunctor`. Они позволяют менять только аннотации дерева, не затрагивая его логическое содержимое. Это важно, поскольку одно и то же дерево проходит через несколько представлений: сначала оно не аннотировано, затем содержит технические аннотации `Z3`, а в конце содержит λ -термы. Экземпляры `Buildable` используются для единообразного вывода формул, секвенций, правил и DOT-графов.

3.3 Конвейер доказательства

Доказательство состоит из двух крупных этапов. Сначала исходная формула IPL преобразуется процедурой клаузификации в r -секвенцию $R, X \Rightarrow g$, где R — множество плоских клауз, X — множество импликационных клауз вида $(a \rightarrow b) \rightarrow c$, а g — атомарная цель. Затем к этой r -секвенции применяется SAT-ориентированный алгоритм `proveR`, описанный Фиорентини [9]. Именно этот алгоритм лежит в основе реализованной функции `sarrow`: он либо строит вывод в $C^\rightarrow$, либо завершает работу конечной контрмоделью Крипке.

Схема алгоритма `proveR` из статьи приведена на рисунке 48. В ней используется один инкрементальный SAT-решатель s , множество интерпретаций W для построения контрмодели и выученная плоская клауза φ .

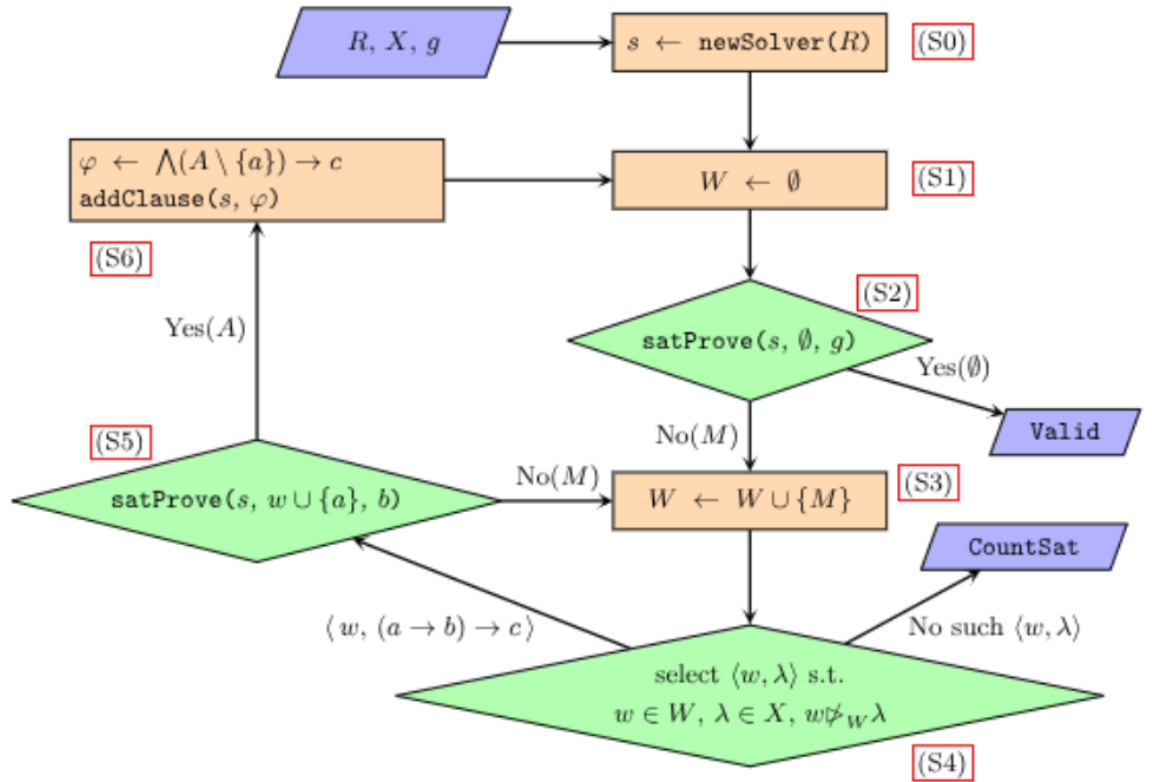


Рисунок 48 — Схема алгоритма `proveR` с перезапуском

Фазы алгоритма на рисунке 48 имеют следующий смысл:

- **S0** — создаётся инкрементальный SAT-решатель $s \leftarrow \text{newSolver}(R)$, содержащий все плоские клаузы из R ;
- **S1** — начинается новая итерация главного цикла, множество интерпретаций контрмодели очищается: $W \leftarrow \emptyset$;

- **S2** — проверяется классическая выводимость цели: $\text{satProve}(s, \emptyset, g)$. Ответ $\text{Yes}(\emptyset)$ завершает алгоритм результатом Valid , а ответ $\text{No}(M)$ запускает построение контрмодели;
- **S3** — найденная интерпретация добавляется в множество миров: $W \leftarrow W \cup \{M\}$;
- **S4** — выбирается пара $\langle w, \lambda \rangle$, где $w \in W$, $\lambda = (a \rightarrow b) \rightarrow c \in X$, и мир w не реализует λ относительно W . Если такой пары нет, алгоритм возвращает CountSat ;
- **S5** — проверяется классическая выводимость $R(s), w, a \vdash_{cpl} b$. При ответе $\text{No}(M)$ внутренний цикл продолжается с новым миром, при ответе $\text{Yes}(A)$ найдено множество допущений для обучения;
- **S6** — строится выученная клауза $\varphi = \bigwedge(A \setminus \{a\}) \rightarrow c$, она добавляется в решатель командой $\text{addClause}(s, \varphi)$, после чего главный цикл перезапускается.

Псевдокод процедуры `proveR`, соответствующий этой схеме, приведён в приложении Д.

Центральная функция `prove` связывает эти фазы в единый конвейер: строит начальную секвенцию, запускает клаузификацию, подготавливает Z3, вызывает поиск вывода в $C^{\rightarrow}$ и в случае успеха запускает построение LJТ-поддеревьев, аннотирование, редукцию и визуализацию. Основной листинг этой функции приведён в приложении Г.

Клаузификация

На первой фазе исходная формула F преобразуется в начальную секвенцию с целью $\$$:

$$F \rightarrow \$ \Rightarrow \$.$$

Функция `clausify` применяет правила `AsImpl`, `AsFlat`, `MakeImpl`, `LeftDs`, `RightCs`, `Uncurry`, `Aliasing` и `ImplImpliesFlat`. Результатом является не только итоговая секвенция `Intuit_`, но и дерево `ClausificationRule_`, фиксирующее все преобразования.

Листинг 18 задаёт порядок, в котором процедура клаузификации пытается применить правила к текущей секвенции.

Листинг 18: Список правил клаузификации `clausificationRules`

```
1 clausificationRules :: [ClausificationRuleSignature]
2 clausificationRules = [ applyFinishClausification,
3                         applyAsImpl,
4                         applyAsFlat,
5                         applyLeftDs,
6                         applyRightCs,
7                         applyUncurry,
8                         applyAliasing,
9                         applyMakeImpl
10                        ]
```

Листинг 19: Функция клаузификации `clausify`

```
1 clausify :: UnclassifiedII -> State ClausificationState_
2           ClausificationRuleII
3 clausify ucseq = do
4   let appliedRules = map ($ ucseq) clausificationRules
5   fromJust $ asum appliedRules
```

Листинг 19 показывает диспетчер: функция строит список возможных применений правил и выбирает первое успешное. Несмотря на использование `map`, все правила не применяются заранее: благодаря ленивым вычислениям Haskell элементы списка `appliedRules` вычисляются только по мере требования со стороны `asum`, поэтому фактически проверяются правила до первого успешного применения.

Листинг 20: Правило клаузификации `applyAsImpl`

```
1 applyAsImpl :: ClausificationRuleSignature
2 applyAsImpl ucseq@(Unclassified flats impls (uc : ucs) goal) = do
3   impl <- fromFormula $ annotated uc
4   let sequent = Unclassified flats (Annotated () impl : impls) ucs goal
5   return $ do
6     rule <- clausify sequent
7     return $ AsImpl ucseq rule (annotated uc) impl []
8 applyAsImpl _ = Nothing
```

Листинг 20 показывает пример правила `AsImpl`: если очередная формула уже распознаётся как импликационная клауза, она переносится из зоны U в зону X , а в дереве сохраняется соответствующий узел правила.

После клаузификации необходимо подготовить Z3. Для этого все атомы, встреченные в плоских и импликационных клаузах, отображаются в булевы переменные Z3, а плоские клаузы добавляются в инкрементальный решатель.

Листинг 21: Добавление плоской клаузы в Z3: addClause

```

1 addClause :: Solver_ -> Flat_ Z3.AST -> IO ()
2 addClause (Solver context solver universe) (Flat cs ds) = do
3   csAST <- Z3.mkAnd context $ annotation <$> cs
4   dsAST <- Z3.mkOr context $ annotation <$> ds
5   clauseAST <- Z3.mkImplies context csAST dsAST
6
7   Z3.solverAssertCnstr context solver clauseAST

```

Листинг 21 показывает, как плоская клауза кодируется в Z3 как классическая импликация между конъюнкцией посылок и дизъюнкцией заключений.

На этой фазе дерево ещё не содержит λ -термов. Аннотация используется технически: каждому атому сопоставляется AST-узел Z3, чтобы далее можно было формулировать классические запросы к решателю. Полный фрагмент подготовки решателя находится в основном листинге функции prove в приложении Г.

Поиск в исчислении $C^{\rightarrow}$

На второй фазе выполняется поиск вывода в исчислении $C^{\rightarrow}$. Функция carrow сначала проверяет, следует ли цель из текущего множества плоских клауз. Если да, строится узел CPL0. Если нет, полученная модель используется как первый мир кандидатной контрмодели, и запускается внутренний цикл.

Внутренний цикл выбирает импликационную клаузу, нарушенную в одном из миров контрмодели, и проверяет с помощью Z3 классическую задачу $R, w, a \vdash b$. При успехе из ядра невыполнимости извлекается множество допущений и строится новая плоская клауза; она добавляется в решатель, после чего основной цикл перезапускается. При неуспехе модель Z3 добавляется как новый мир контрмодели. Если нарушенных импликационных клауз больше нет, построенная структура миров является контрмоделью Крипке.

Листинг 22 соответствует внешнему циклу proveR: он либо завершает доказательство правилом CPL0, либо добавляет выученную клаузу и рекурсивно перезапускает поиск.

Листинг 22: Основной цикл поиска вывода в $C^{\rightarrow}$

```

1  carrow :: Intuit_ () AST -> Solver_ ->
2      IO (Either (CounterModel_ AST) (CArrowRule_ () AST))
3  carrow iseq@(Intuit flats impls goal) solver = do
4      result <- satProve solver [] goal
5      case result of
6          Yes _ -> return $ Right $ CPL0 iseq (Classic flats [] goal)
7          No model -> do
8              result <- innerLoop iseq (newCounterModel model) solver
9              case result of
10                  Left counterModel -> return $ Left counterModel
11                  Right (assumptions, learnedClause) -> do
12                      let (Impl a b c) = learnedClause
13                      let classicSequent =
14                          Classic flats (addAnnotation () <$> a : assumptions)
15                          (addAnnotation () b)
16                      let newClause = Flat (List.delete a assumptions) [c]
17                      addClause solver newClause
18
19                      let intuitSequent = Intuit (Annotated () newClause : flats)
20                          impls goal
21                      result <- carrow intuitSequent solver
22
23                  case result of
24                      Left counterModel -> return $ Left counterModel
25                      Right proof ->
26                          return $ Right $ CPL1 iseq classicSequent proof newClause
27                          learnedClause

```

Листинг 23 соответствует внутреннему циклу: он уточняет множество миров контрмодели до тех пор, пока не будет найдена нарушенная импликационная клауза, из которой можно получить новое ограничение.

Построение LJT-поддеревьев

Классические подзадачи, возникающие в узлах CPL0 и CPL1, передаются в Z3 как SAT-запросы. Теоретически из решателя можно получать доказательства невыполнимости или ядра невыполнимости, однако их дальнейшая интерпретация усложняет реализацию: такие объекты зависят от внутреннего представления решателя и плохо соотносятся с правилами исходного исчисления. В данной работе выбран более простой и прозрачный путь: после того как решатель подтвердил классическую выводимость подзадачи, для неё строится явное доказательство в

Листинг 23: Внутренний цикл построения контрмодели

```

1 innerLoop ::
2   Intuit_ () AST ->
3   CounterModel_ AST ->
4   Solver_ ->
5   IO (Either (CounterModel_ AST) ([Annotated_ AST Atom_], Impl_ AST))
6 innerLoop iseq@(Intuit flats impls goal) counterModel solver = do
7   let selectedWorld =
8       asum [do sw <- selectWorld counterModel i; return (i, sw)
9           | (Annotated () i) <- impls]
10  case selectedWorld of
11    Just (learned@(Impl a b c), (World worldId as)) -> do
12      result <- satProve solver (Set.toList $ Set.insert a as)
13      (addAnnotation () b)
14    case result of
15      Yes core -> return $ Right (core, learned)
16      No model -> innerLoop iseq (addWorld counterModel worldId model)
17      solver
18    Nothing -> return $ Left counterModel

```

упрощённой системе LJТ. Благодаря специальной форме классических секвенций этот алгоритм поиска доказательства получается небольшим, а полученное дерево сразу пригодно для последующего аннотирования λ -термами.

На третьей фазе доказательство расширяется LJТ-поддеревьями. Узлы CPL0 и CPL1 содержат классические секвенции, которые затем доказываются функцией `ljt`. В новой реализации эта фаза отделена от самого CEGAR-поиска: `carrow` отвечает за построение линейной структуры $C^{\rightarrow}$, а `extendProof` добавляет к ней узлы ExCPL0 и ExCPL1 с деревьями LJТ.

Листинг 24: Добавление LJТ-поддеревьев к доказательству $C^{\rightarrow}$

```

1 extendProof :: CArrowRule_ () () -> CArrowRule_ () ()
2 extendProof (CPL0 iseq cseq) =
3   ExCPL0 iseq (ljt cseq) []
4 extendProof (CPL1 iseq cseq rule newFlat learnedImpl) =
5   ExCPL1 iseq (ljt cseq) (extendProof rule) newFlat learnedImpl []
6 extendProof _ = undefined

```

Листинг 24 показывает, как к каждому узлу CPL0 или CPL1 добавляется LJТ-доказательство соответствующей классической секвенции.

Листинг 25: Функция построения LJТ-доказательства ljt

```

1 ljt :: Classic_ () () -> LJTRule_ () ()
2 ljt cseq@(Classic flats assumptions goal) =
3   fromJust $ asum (map ($ cseq) [applyAxiom, applyCs, applyDs])

```

Листинг 25 задаёт порядок применения трёх правил LJТ-поддерева: аксиомы, сокращения левой конъюнкции и разбиения дизъюнкции.

Поскольку в фазу LJТ попадают не произвольные интуиционистские сечвенции, а классические подзадачи специального вида $R, A \vdash g$, полная система LJТ Дайкхоффа [7] используется в существенно суженном виде. Здесь R состоит только из плоских клауз

$$a_1 \wedge \dots \wedge a_n \rightarrow d_1 \vee \dots \vee d_m,$$

а A содержит только атомарные допущения. Поэтому для построения дерева достаточно следующих рассуждений:

- если цель уже содержится среди атомарных допущений, применяется аксиома;
- если есть клауза без левой части $\top \rightarrow d_1 \vee \dots \vee d_m$, используется левое правило для дизъюнкции: доказательство ветвится по всем возможным дизъюнктам;
- если левая часть клаузы содержит атом, уже известный в допущениях, этот атом можно удалить из левой части. Логически этот шаг соответствует каррированию плоской клаузы и последующему *modus ponens*.

В теоретическом описании последний пункт удобно рассматривать как два правила: сначала клауза

$$(a_1 \wedge \dots \wedge a_i \wedge \dots \wedge a_n) \rightarrow D$$

переписывается в форму

$$a_i \rightarrow ((a_1 \wedge \dots \wedge a_{i-1} \wedge a_{i+1} \wedge \dots \wedge a_n) \rightarrow D),$$

а затем из наличия a_i в допущениях по modus ponens получается редуцированная клауза

$$(a_1 \wedge \dots \wedge a_{i-1} \wedge a_{i+1} \wedge \dots \wedge a_n) \rightarrow D.$$

В реализации эти два шага объединены в одно правило `ReduceConjunction`. Поэтому тип дерева LJT содержит три конструктора: `Axiom`, `SplitDisjunction` и `ReduceConjunction`.

Листинг 26: Три правила LJT, используемые для классических подзадач

```

1 data LJTRule_ a c =
2   Axiom
3     (Classic_ a c)
4   |
5   SplitDisjunction
6     (Classic_ a c)
7     [LJTRule_ a c]
8     (Flat_ ())
9     [Substitution_]
10  |
11  ReduceConjunction
12    (Classic_ a c)
13    (LJTRule_ a c)
14    (Flat_ ())
15    Atom_
16    [Substitution_]

```

Листинг 26 показывает тип дерева LJT, который используется только для классических подзадач, возникающих внутри $C^{\rightarrow}$. В конструкторах правил сохраняются подстановки, которые затем выводятся в DOT-представлении рядом с соответствующими узлами доказательства.

Далее приведены локальные функции из блока `where` функции `ljt`. Аксиома завершает ветвь доказательства. В терминах секвенций она соответствует правилу и в коде проверяется простым условием `goal 'elem' assumptions`.

$$\frac{}{R, A, g \vdash g} Ax$$

Рисунок 49 — Аксиома LJT-поддерева в реализации

Листинг 27 показывает локальную реализацию аксиомы: ветвь закрывается, когда цель уже присутствует среди допущений.

Листинг 27: Локальная проверка аксиомы в LJТ-поддереве

```

1 applyAxiom :: Classic_ () () -> Maybe (LJTRule_ () ())
2 applyAxiom cseq@(Classic flats assumptions goal) =
3   if goal `elem` assumptions
4   then Just (Axiom cseq)
5   else Nothing

```

Правило `SplitDisjunction` применяется к плоской клаузе с пустой левой частью. Такая клауза имеет вид $\top \rightarrow d_1 \vee \dots \vee d_m$, то есть в текущем контексте истинна дизъюнкция. Чтобы доказать цель g , нужно показать её из каждого возможного дизъюнкта:

$$\frac{R, A, d_1 \vdash g \quad \dots \quad R, A, d_m \vdash g}{R, A, d_1 \vee \dots \vee d_m \vdash g} \vee_L$$

Рисунок 50 — Разбиение LJТ-поддерева по дизъюнкции

Листинг 28: Локальное разбиение по правой части плоской клаузы

```

1 applyDs :: Classic_ () () -> Maybe (LJTRule_ () ())
2 applyDs cseq@(Classic flats assumptions goal) = do
3   found@(Annotated () (Flat _ ds)) <- maybeFoundDs
4   return $
5     SplitDisjunction
6       cseq
7       (ljt <$> [
8         Classic (List.delete found flats) (addAnnotation () d :
9           assumptions) goal
10        | d <- ds
11      ])
12   (annotated found) []
13 where
14   maybeFoundDs = List.find dsPred flats
15   dsPred (Annotated () (Flat [] _)) = True
16   dsPred _ = False

```

Листинг 28 показывает ветвление доказательства по каждому атому из правой части плоской клаузы.

Правило `ReduceConjunction` применяется, когда в левой части плоской клаузы есть атом, уже находящийся в списке допущений. Пусть в R есть клауза $(a_i \wedge C) \rightarrow D$, а среди допущений есть a_i . Тогда вместо этой клаузы можно продолжить доказательство с более простой клаузой $C \rightarrow D$: В терминах исход-

$$\frac{R, C \rightarrow D, A, a_i \vdash g}{R, (a_i \wedge C) \rightarrow D, A, a_i \vdash g} \text{reduce}$$

Рисунок 51 — Сокращение левой конъюнкции в LJТ-поддереве

ных правил это сокращение соответствует каррированию $(a_i \wedge C) \rightarrow D$ и одному применению *modus ponens* к допущению a_i .

Листинг 29: Локальное объединение каррирования и *modus ponens*

```

1  applyCs :: Classic_ () () -> Maybe (LJTRule_ () ())
2  applyCs cseq@(Classic flats assumptions goal) = do
3    (atom, flat@(Annotated () (Flat cs ds))) <- maybeFoundCs
4    let newClause = Flat (List.delete (reannotate (const ()) atom) cs) ds
5    let newSequent =
6      Classic (Annotated () newClause : List.delete flat flats)
7      assumptions goal
7  return
8    (ReduceConjunction
9      cseq
10     (ljt newSequent)
11     (annotated flat)
12     (annotated atom) [])
13  where
14    maybeFoundCs = List.find (uncurry csPred)
15      [(a', r') | a' <- assumptions, r' <- flats]
16    csPred atom (Annotated () (Flat cs _)) =
17      annotated atom `elem` map annotated cs

```

Листинг 29 показывает реализованное сокращение: найденный в допущениях атом удаляется из левой части плоской клаузы.

Порядок применения правил в функции *ljt* выбран так, чтобы сначала закрывать уже доказанные цели, затем сокращать левые части плоских клауз по имеющимся атомам и только после этого ветвиться по дизъюнкции:

$$\text{applyAxiom} \rightarrow \text{applyCs} \rightarrow \text{applyDs}.$$

Такой порядок уменьшает размер классической подзадачи до ветвления и тем самым делает получаемое LJТ-поддерево компактнее.

Аннотирование λ -термами

На четвёртой фазе выполняется аннотирование λ -термами. Функция `annotateClausification` проходит по объединённому дереву клаузификации и $C^{\rightarrow}$ в обратном направлении, восстанавливая термовые преобразования. Для LJT-поддеревьев используется функция `annotateLJT`; для узлов $C^{\rightarrow}$ — функция `annotateCArrow`. В окружении `Environment` хранятся свежие имена термов и соответствия между формулами и уже введёнными переменными.

Листинг 30: Переход от клаузификации к аннотированному $C^{\rightarrow}$

```
1  annotateClausification (StartCArrow useq carrowRule) = do
2    annotatedCArrow <- annotateCArrow carrowRule
3    let (Intuit flats impls goal) = rootCArrow annotatedCArrow
4
5    return $
6      StartCArrow
7        (Unclassified flats impls [] goal)
8      annotatedCArrow
```

Листинг 30 показывает граничный случай аннотирования: после окончания клаузификации управление передаётся дереву $C^{\rightarrow}$.

Листинг 31: Аннотирование правила ExCPL0: `annotateCArrow`

```
1  annotateCArrow :: CArrowRule_ () () -> State Environment (CArrowRule_
2    Lambda_ ())
3  annotateCArrow (ExCPL0 (Intuit _ impls _) ljtRule _) = do
4    annotatedLJT <- annotateLJT ljtRule
5    let (Classic flats _ goal) = rootLJT annotatedLJT
6    implAnnotations <- mapM getTerm (annotated <$> impls)
7    let annotatedImpls =
8      [Annotated ann (annotated impl) | (impl, ann) <- zip impls
9        implAnnotations]
10
11    return $
12      ExCPL0
13        (Intuit flats annotatedImpls goal)
14      annotatedLJT []
```

Листинг 31 показывает, как результат LJT-доказательства переносится в аннотированный узел ExCPL0.

Редукция и визуализация

На пятой фазе итоговый терм нормализуется функцией `Lambda.reduce`. В текущей реализации также печатается промежуточный DOT-граф дерева клаузификации до запуска решателя. После успешного поиска вывода полное аннотированное дерево доказательства выводится в формате Graphviz DOT, а в случае невыводимости вместо доказательства печатается DOT-граф контрмодели Крипке.

Листинг 32: Фрагмент функции редукции `reduce`

```
1 reduce :: Lambda_ -> Lambda_
2 reduce (Abstraction [] body) = reduce body
3 reduce (Abstraction capture body) = Abstraction capture $ reduce body
4 reduce (Const body) = Const $ reduce body
5 reduce v@(Variable _) = v
6 reduce cs@(Case e cases) =
7   case reduce e of
8     Sum c i arg -> if c /= length cases
9                     then error "Wrong case"
10                    else let (capture, body) = cases !! (i - 1)
11                          substitution = Substitution capture arg
12                          in reduce $ substitute body substitution
13   t -> Case t [(capture, reduce body) | (capture, body) <- cases]
```

Листинг 32 показывает фрагмент нормализации λ -термов, включая редукцию разбора случаев для суммы.

Листинг 33: Фрагмент функции визуализации `buildDotClausify`

```
1 buildDotClausify ::
2   (BuildableDotAnnotation a, BuildableDotAnnotation c) =>
3   Int -> ClausificationRule_ a c -> Builder
4 buildDotClausify rootNodeId (AsImpl c r _ _ subs) =
5   rootNodeId |+ " [label=\"\" <> buildDot c <> "|{" <>
6   joinBy "\\n" (buildDot <$> subs) <> "}\"\\n" +|
7   rootNodeId |+ " -> " +| (rootNodeId + 1) |+ " [label=AsImpl]\\n\" <>
8   buildDotClausify (rootNodeId + 1) r
9 buildDotClausify rootNodeId (StartArrow c r) =
10  rootNodeId |+ " [label=\"\" <> buildDot c <> "\"\\n\" +|
11  rootNodeId |+ " -> " +| (rootNodeId + 1) |+ " [label=StartArrow]\\n\" <>
12  buildDotCArrow (rootNodeId + 1) r
```

Листинг 33 показывает, как узлы дерева клаузификации превращаются в вершины и рёбра графа Graphviz.

Здесь дерево клаузификации и расширенное дерево $C \rightarrow$ склеиваются в одно дерево, аннотируются, затем все термы редуцируются функцией `Lambda.reduce`. После этого дерево передаётся в `buildDotClausify`, которая строит текстовое представление графа Graphviz. Последовательность основных вызовов приведена в листинге функции `prove` в приложении Г.

3.4 Построение контрмодели

Если формула не является теоремой IPL, процедура строит конечную контр-модель Крипке. В реализации она представлена типами `World` и `CounterModel_`, приведёнными в листингах 14 и 15. Мир `World` хранит числовой идентификатор и множество атомов, истинных в этом мире. Значение `CounterModel_` хранит все построенные миры в отображении `worlds`, а отношение достижимости поддерживает в двух направлениях: `reachableTo` задаёт список предков мира, а `reachableFrom` — список его непосредственных потомков.

Такое представление удобно для двух операций, которые выполняются во внутреннем цикле алгоритма. Во-первых, по выбранному миру нужно быстро получить всех его потомков и проверить условие истинности импликационной клаузы. Во-вторых, при добавлении нового мира необходимо сохранить атомы, полученные из новой модели Z3, и зафиксировать, что этот мир становится потомком выбранного мира.

Листинг 34: Создание начальной контрмодели `newCounterModel`

```
1 newCounterModel :: Ord a => [Annotated_ a Atom_] -> CounterModel_ a
2 newCounterModel world = CounterModel {
3   reachableTo = Map.singleton 0 [],
4   reachableFrom = Map.singleton 0 [],
5   worlds = Map.singleton 0 (World 0 (Set.fromList world))
6 }
```

Листинг 34 показывает создание корневого мира с номером 0. Он строится из первой модели Z3: список истинных атомов переводится во множество, а списки предков и потомков инициализируются пустыми.

Листинг 35 задаёт единственную операцию расширения контрмодели. Новый мир получает следующий свободный номер, сохраняет атомы из новой модели

Листинг 35: Добавление нового мира в контрмодель addWorld

```
1 addWorld :: Ord a => CounterModel_ a -> Int -> [Annotated_ a Atom_] ->
  CounterModel_ a
2 addWorld counterModel worldId as =
3   counterModel {
4     reachableTo =
5       Map.insert newWorldId reachableToNewWorld (reachableTo
6         counterModel),
7     reachableFrom =
8       Map.insert newWorldId [] $
9         List.foldr (Map.update (Just . (:) newWorldId))
10          (reachableFrom counterModel) reachableToNewWorld,
11     worlds = Map.insert newWorldId newWorld (worlds counterModel)
12   }
13   where
14     newWorldId = Map.size $ worlds counterModel
15     newWorld = World newWorldId (Set.fromList as)
16     reachableToNewWorld = worldId : (reachableTo counterModel ! worldId)
```

Z3 и становится потомком мира worldId. Список reachableToNewWorld содержит выбранный мир и всех его предков; поэтому для каждого из них обновляется отображение reachableFrom.

Листинг 36: Поиск мира, нарушающего импликационную клаузу selectWorld

```
1 selectWorld :: Ord a => CounterModel_ a -> Impl_ a -> Maybe (World a)
2 selectWorld counterModel impl@(Impl a b c) =
3   selectWorlds' [0..(Map.size (worlds counterModel) - 1)]
4   where
5     selectWorlds' [] = Nothing
6     selectWorlds' (worldId:t) =
7       if condition
8       then selectWorlds' t
9       else Just world
10    where
11      world@(World _ as) = worlds counterModel ! worldId
12      reachableFromWorlds =
13        map (worlds counterModel !) (reachableFrom counterModel !
14          worldId)
15      condition =
16        Set.member a as ||
17        Set.member b as ||
18        Set.member c as ||
19        any (reachableFromPred impl) reachableFromWorlds
```


Листинг 36 показывает основной способ чтения контрмодели. Функция перебирает уже построенные миры и ищет первый мир, в котором импликационная клауза $(a \rightarrow b) \rightarrow c$ ещё не реализована. Если такого мира нет, текущая структура миров уже является контрмоделью для исходной формулы.

Листинг 37: Предикат для потомков выбранного мира

```
1 reachableFromPred (Impl a b _) (World _ as) =
2   Set.member a as && not (Set.member b as)
```

Листинг 37 уточняет последнюю часть проверки из `selectWorld`. Если среди потомков уже есть мир, где истинно a и ложно b , то требуемый свидетель для импликации $a \rightarrow b$ уже найден, и выбранный мир не нарушает рассматриваемую импликационную клаузу.

Листинг 38: Вывод контрмодели в формате DOT

```
1 instance Buildable (CounterModel_ a) where
2   build (CounterModel reachableTo reachableFrom worlds) =
3     "digraph {\n" <>
4     "  graph [rankdir=BT]\n" <>
5     "  node [shape=record]\n" <>
6     indentF 2 (unlinesF $ Map.elems worlds) <>
7     indentF 2 (Foldable.fold [
8       ancestor |+ " -> " +| i |+ "\n"
9       | (i, (ancestor: _)) <- Map.toAscList reachableTo]) <>
10    "}"
```

Листинг 38 показывает интерфейс визуализации: контрмодель преобразуется в граф Graphviz. В вершинах выводятся миры с наборами истинных атомов, а рёбра строятся по отношению достижимости.

3.5 Примеры использования

Исполняемый модуль `app/Main.hs` читает формулу из файла, путь к которому передаётся единственным аргументом командной строки. Примеры входных формул находятся в каталоге `examples/formulas`. Для части этих формул заранее сохранены результаты работы программы: редуцированные термы для выводимых формул и изображения контрмоделей для невыводимых. Эти файлы

находятся в каталоге `examples/terms` и имеют те же имена, что и соответствующие входные данные.

Листинг 39: Запуск программы на файле с формулой

```
1 stack exec diploma-exe -- examples/formulas/complex-and-impl-chain.txt
```

Листинг 39 показывает общий способ запуска. После чтения файла программа разбирает формулу, запускает конвейер доказательства и печатает DOT-представление результата. Для валидных формул из этого результата можно извлечь редуцированный λ -терм.

Листинг 40: Входная формула `complex-and-impl-chain.txt`

```
1 (((a /\ (b /\ c) /\ d) => c) => a) => a
```

Листинг 40 задаёт формулу, в которой из доказательства импликации к a нужно передать функцию, извлекающую компоненту c из конъюнкции.

Листинг 41: Редуцированный терм для `complex-and-impl-chain.txt`

```
1 \_23.(_23(\_7.Proj(4, 3, _7)))
```

Листинг 41 показывает результат аннотирования и редукции: терм применяет входной аргумент к функции, которая проецирует третью компоненту из произведения.

Листинг 42: Входная формула `complex-impl-disjunction.txt`

```
1 (((a \/ (a => b))) => b) => b
```

Листинг 42 демонстрирует пример с дизъюнкцией: для доказательства b достаточно уметь обработать либо готовое доказательство a , либо функцию $a \rightarrow b$.

Листинг 43: Редуцированный терм для `complex-impl-disjunction.txt`

```
1 \_5.(_5Sum(2, 2, (\_4.(_5Sum(2, 1, _4)))))
```

Листинг 43 показывает, что в терме используется вложение в сумму: программа передаёт обработчику правый вариант дизъюнкции, а внутри него при необходимости строит левый вариант.

Если формула не является теоремой IPL, вместо терма строится конечная контрмодель Крипке. Для закона исключённого третьего входной файл содержит формулу, приведённую в листинге 44.

Листинг 44: Входная формула `invalid-excluded-middle.txt`

```
1 A \ / -A
```

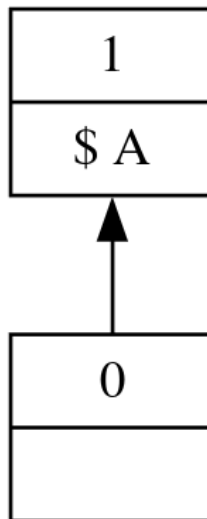


Рисунок 52 — Контрмодель для закона исключённого третьего

На рисунке 52 показана контрмодель, в которой корневой мир не вынуждает ни A , ни $\neg A$, поэтому дизъюнкция $A \vee \neg A$ не является интуиционистски общезначимой.

Листинг 45: Входная формула `invalid-peirce.txt`

```
1 ((A => B) => A) => A
```

Листинг 45 задаёт схему Пирса, которая выводима в классической логике, но не выводима в IPL.

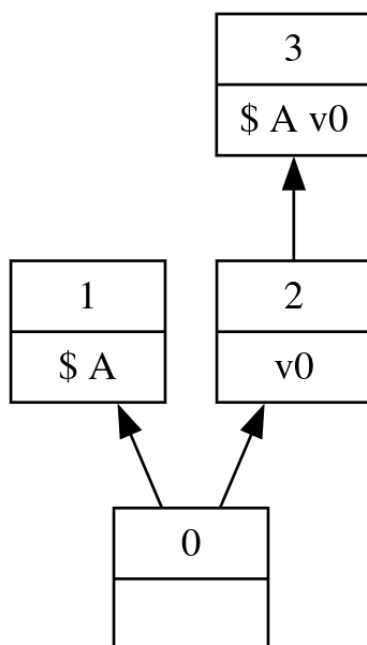


Рисунок 53 — Контрмодель для схемы Пирса

Рисунок 53 показывает соответствующую контрмодель: в разных потомках корневого мира реализуются разные атомарные оценки, из-за чего формула $((A \rightarrow B) \rightarrow A) \rightarrow A$ не вынуждается в корне.

4 Аналитическая часть

В данном разделе приведены результаты измерения производительности разработанного прототипа и сравнение с аналогом `intuitR` [5] на семействах бенчмарков `syj201`, `syj207` и `fresh-tail`.

4.1 Методика измерений

Для разработанного доказателя добавлен отдельный исполняемый модуль `diploma-bench`. Он использует тот же разбор формул и тот же конвейер доказательства, что и `diploma-ехе`, но вместо печати дерева вывода возвращает структурированную статистику по трём фазам:

- `clausification_ms` — построение начальной секвенции и клаузификация;
- `proving_ms` — подготовка Z3, добавление начальных плоских клауз и запуск поиска вывода в $C^{\rightarrow}$;
- `annotation_ms` — построение LJТ-поддеревьев, объединение дерева клаузификации с деревом доказательства и извлечение λ -терма.

Измерение выполнялось на локальной машине под управлением Linux с GHC 9.10.3 и Z3, запускаемыми через `stack`. Время фиксировалось монотонным системным таймером в миллисекундах. Исполняемые файлы, использующие Z3, собраны с параметром RTS `-N1`: при многопоточном RTS (`-N`) вызовы низкоуровневого API Z3 приводили к нестабильному завершению процесса.

Листинг 46: Запуск бенчмарка разработанного доказателя

```
1 stack exec diploma-bench -- benchmarks/syj201/formulas
```

Для сравнения с `intuitR` использовался внешний исполняемый файл `intuitR`, принимающий формулы в формате TPTP. Для каждого значения параметра N генерировался файл `syj201-nXX.p`; `intuitR` самостоятельно выполняет клаузификацию и доказательство, печатая время двух фаз в секундах с точностью до миллисекунды.

Листинг 47: Генерация семейства syj201 и запуск intuitR

```
1 stack exec syj201-gen -- 3 > benchmarks/syj201/formulas/syj201-n03.txt
2 intuitR benchmarks/syj201/intuitr-formulas/syj201-n03.p
```

4.2 Семейство бенчмарков syj201

Основной набор экспериментов — семейство формул syj201 из библиотеки TRTP (ILTP, семейство SYJ201). Для параметра N строится формула с $m = 2N + 1$ атомами $p_1, \dots, p_m$. Левая часть состоит из конъюнкции импликаций

$$(p_i \leftrightarrow p_{i+1}) \rightarrow c(N), \quad i = 1, \dots, m,$$

где индексация $i + 1$ берётся по модулю m , а $c(N) = p_1 \wedge \dots \wedge p_m$. Заключение совпадает с $c(N)$. Для $N = 1$ получается исходный пример syj201.txt; генератор syj201-gen позволяет получать формулы для произвольного N в синтаксисе прототипа и в формате TRTP для intuitR.

Дополнительно на каталоге examples/formulas (18 формул, включая syj201.txt) были получены средние времена по фазам. Для выводимых формул среднее полное время составило 15,581 мс, при этом основной вклад даёт фаза доказательства (15,251 мс); клаузификация и аннотирование занимают менее 0,05 мс в среднем.

4.3 Результаты для разработанного доказателя

На семействе syj201 для $N = 1, \dots, 10$ прототип успешно доказал все формулы. Время резко растёт с параметром N : от 17 мс при $N = 1$ до примерно 977 с (≈ 16 мин) при $N = 10$. Результаты приведены в таблице 1 и на рисунке 54.

Из таблицы 1 видно, что основной вклад даёт фаза доказательства: при $N = 10$ она составляет более 99,99% полного времени. Клаузификация и аннотирование остаются малыми на всём диапазоне N , однако рост proving_ms носит сверхлинейный характер и указывает на структурную, а не чисто инженерную причину замедления.

Таблица 1 — Время фаз разработанного доказателя на семействе `syj201`

N	Результат	Клауз., мс	Доказ., мс	Аннот., мс	Всего, мс
1	valid	0.274	16.293	0.086	16.653
2	valid	0.190	54.285	0.180	54.656
3	valid	0.390	184.823	1.150	186.362
4	valid	0.417	601.076	1.729	603.223
5	valid	0.789	1334.431	3.439	1338.659
6	valid	0.774	13106.809	5.797	13113.380
7	valid	1.152	19767.165	9.816	19778.133
8	valid	1.246	46416.473	15.934	46433.652
9	valid	2.189	179700.170	24.591	179726.950
10	valid	2.721	976877.759	30.535	976911.016

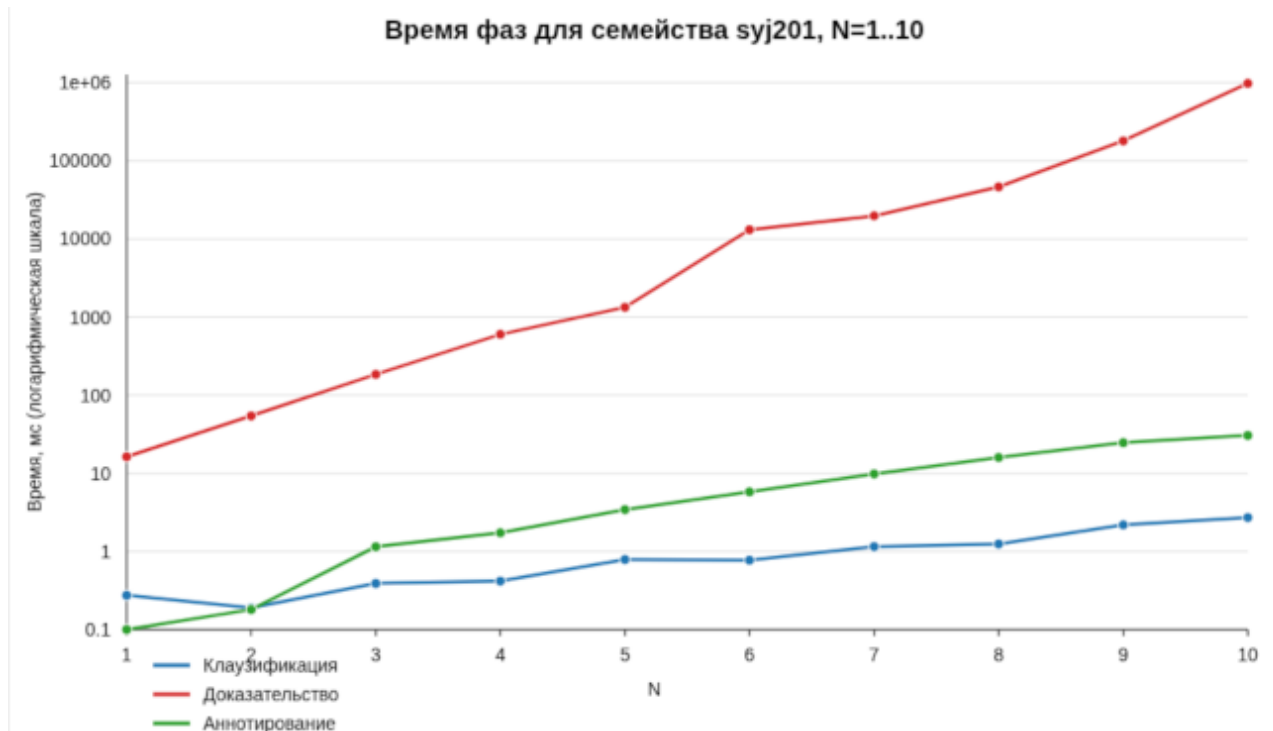


Рисунок 54 — Время фаз разработанного доказателя для $N = 1, \dots, 10$

На рисунке 54 показано распределение времени по фазам; доминирует кривая доказательства, тогда как клаузификация и аннотирование остаются у нижней границы логарифмической шкалы.

4.4 Сравнение с `intuitR`

В качестве аналога использовался `intuitR` — реализация процедуры `proveR` с перезапуском, описанная в конструкторской части. На том же семей-

стве `syj201` оба инструмента доказали все формулы для $N = 1, \dots, 10$. Сводная таблица приведена в таблице 2.

Таблица 2 — Сравнение разработанного доказателя и `intuitR` на `syj201`

N	Прототип	Время, мс	intuitR	Клауз.+доказ., мс	Wall, мс
1	valid	16.653	valid	0.000	12.360
2	valid	54.656	valid	1.000	12.287
3	valid	186.362	valid	1.000	12.236
4	valid	603.223	valid	1.000	12.948
5	valid	1338.659	valid	2.000	12.784
6	valid	13113.380	valid	3.000	12.670
7	valid	19778.133	valid	5.000	12.122
8	valid	46433.652	valid	4.000	12.676
9	valid	179726.950	valid	8.000	22.046
10	valid	976911.016	valid	6.000	12.587

В таблице 2 столбец «Клауз.+доказ.» содержит сумму времени клаузификации и доказательства, которую `intuitR` печатает самостоятельно; столбец «Wall» — полное wall-clock время запуска процесса. `intuitR` выводит время с точностью 0,001 с, поэтому малые значения округляются до нуля.

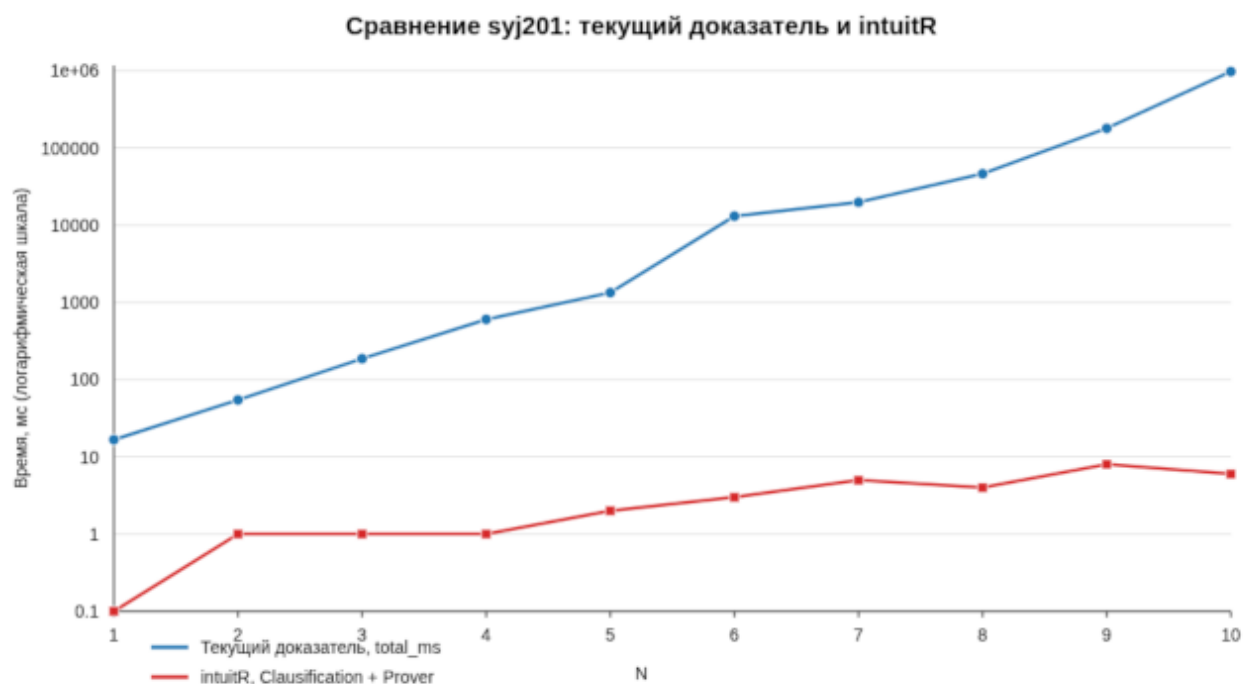


Рисунок 55 — Сравнение полного времени: прототип и `intuitR`

На рисунке 55 сопоставлены полное время прототипа и суммарное время фаз `intuitR`. При одинаковом исходе (формула доказуема для всех N) разрыв достигает нескольких порядков уже при малых N и сохраняется на всём диапазоне.

Предположения о расхождении производительности

Оба инструмента решают одну и ту же задачу проверки выводимости на клаузифицированных τ -секвенциях, однако прототип измеряет полный конвейер с построением дерева клаузификации, дерева доказательства в $C^{\rightarrow}$ и последующим аннотированием. По данным таблицы 1, основной разрыв с `intuitR` формируется в фазе `proving_ms`. Ниже выделены два ключевых фактора; первый требует доработки алгоритма, второй — осознанное архитектурное решение прототипа.

- **Асимптотика клаузификации.** Текущая процедура клаузификации на семействе `syj201` порождает слишком большое число плоских и импликационных клауз относительно параметра N . Это увеличивает размер τ -секвенции, число итераций CEGAR и SAT-запросов к Z3. Рост времени доказательства от 17 мс при $N = 1$ до 977 с при $N = 10$ при почти неизменных клаузификации и аннотировании указывает именно на сверхлинейную «комбинаторику» после клаузификации, а не на постоянные накладные расходы одного вызова решателя. Сокращение числа порождаемых клауз — направление, над которым следует работать в первую очередь.
- **Сохранение истории доказательства.** В отличие от `proveR` с перезапуском, прототип накапливает дерево клаузификации и цепочку узлов CPL0/CPL1 в $C^{\rightarrow}$, а затем объединяет их для аннотирования и визуализации. `intuitR` после добавления выученной клаузы очищает множество интерпретаций W и не хранит полную историю промежуточных шагов. Сохранение истории необходимо для целей работы — извлечения λ -терма и построения DOT-графа — но увеличивает объём работы на каждой итерации CEGAR и объясняет часть отставания от эталона, оптимизированного только под ответ «доказуемо/нет».
- **Дополнительные инженерные факторы.** Классические подзапросы выполняются через Haskell-обёртку Z3 с извлечением `unsat core`; `wall-clock intuitR` (≈ 12 мс) включает старт процесса, а внутренние таймеры округляются до 0,001 с. Эти эффекты не снимают основного разрыва, но могут вносить вклад при малых N .

Таким образом, отставание прототипа от `intuitR` интерпретируется как сочетание плохой асимптотики клаузификации (проблема, требующая алгорит-

мической доработки) и необходимого для прототипа сохранения истории вывода (компромисс ради программного содержания и наглядности). Для количественной проверки потребовались бы счётчики числа клауз после клаузации, длины выводов в $C^{\rightarrow}$ и числа вызовов `satProve` на каждом N .

4.5 Семейство бенчмарков `syj207`

Дополнительно рассмотрено семейство `syj207` (ILTP, SYJ207; схема `psi.p`). Для параметра N задаётся цикл из $m = 2N$ атомов $p_1, \dots, p_m$ и отдельный атом p_0 . Левая часть — конъюнкция импликаций $(p_i \leftrightarrow p_{i+1}) \rightarrow c(N)$ по циклу, где $c(N) = p_1 \wedge \dots \wedge p_m$. Заключение имеет вид $p_0 \vee c(N) \vee \neg p_0$ и не выводимо в IPL (ожидается контрмодель). Генератор `syj207-gen` строит формулы в синтаксисе прототипа и в формате TPTP для `intuitR`. Измерения проводились для $N = 1, \dots, 5$; для $N \geq 6$ время прототипа превышало лимит 600 с на формулу, поэтому серия была остановлена.

Таблица 3 — Время фаз разработанного доказателя на семействе `syj207` ($N = 1, \dots, 5$)

N	Результат	Клауз., мс	Доказ., мс	Аннот., мс	Всего, мс
1	invalid	1.232	36.711	0.000	37.943
2	invalid	2.024	68.896	0.000	70.920
3	invalid	3.293	338.073	0.000	341.367
4	invalid	5.238	132440.380	0.000	132445.618
5	invalid	7.008	8631.909	0.000	8638.917

Прототип корректно находит контрмодели для всех пяти случаев. Время доказательства не монотонно растёт с N : пик при $N = 4$ составляет около 132 с, тогда как при $N = 5$ — 8,6 с. Это согласуется с зависимостью поиска от конкретной структуры клауз, а не только от размера формулы.

Таблица 4 — Сравнение разработанного доказателя и `intuitR` на `syj207` ($N = 1, \dots, 5$)

N	Прототип	Время, мс	intuitR	Клауз.+доказ., мс	Wall, мс
1	invalid	37.943	invalid	0.000	15.124
2	invalid	70.920	invalid	1.000	15.345
3	invalid	341.367	invalid	2.000	14.953
4	invalid	132445.618	invalid	2.000	15.538
5	invalid	8638.917	invalid	5.000	25.558

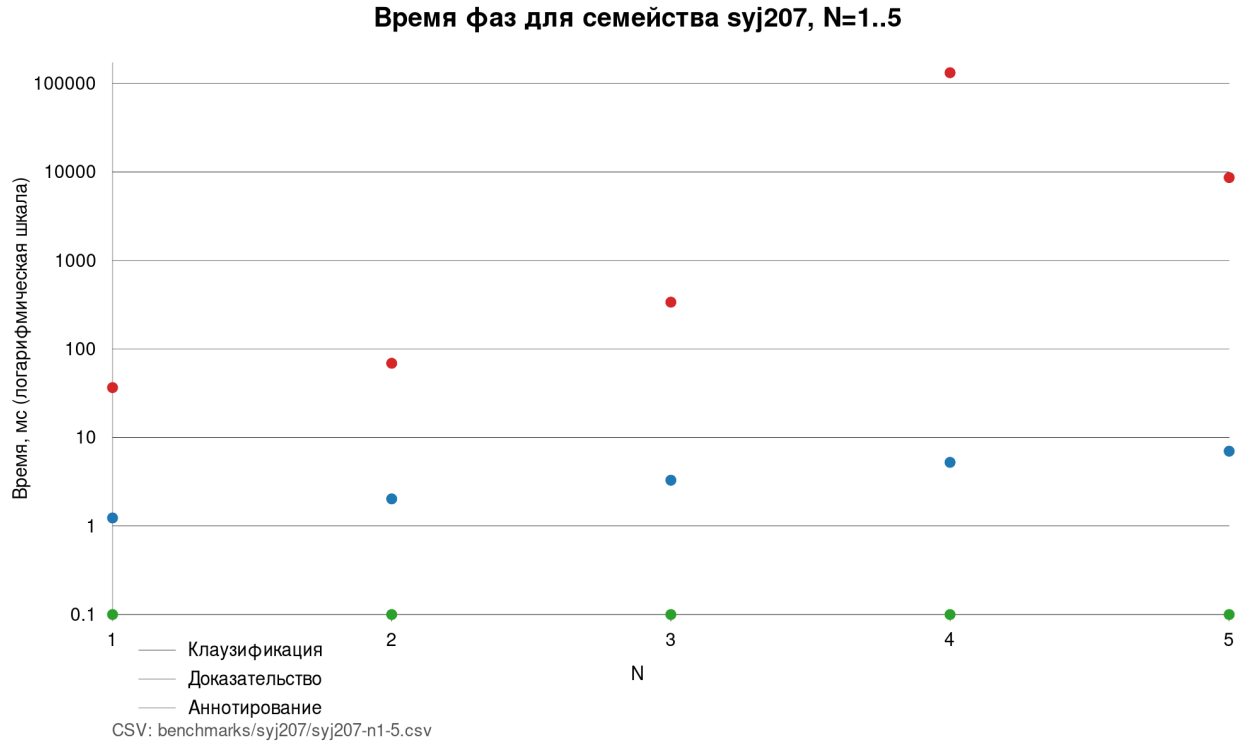


Рисунок 56 — Время фаз прототипа для syj207, $N = 1, \dots, 5$

На таблице 4 оба инструмента согласны по исходу (*invalid*). *intuitR* укладывается в единицы–десятки миллисекунд по внутренним таймерам на всём диапазоне, тогда как прототип на $N = 4$ тратит более двух минут на поиск контрмодели.

4.6 Семейство формул *fresh-tail*

Для проверки поведения на валидных формулах с регулярно растущей структурой импликаций построено семейство *fresh-tail*. Базой служит выводимая формула syj201 при $N = 1$; поверх неё n раз накладывается схема

$$((\dots) \Rightarrow p_n) \Rightarrow p_n,$$

где каждый p_n — свежий атом, не встречающийся ранее (начиная с d). Генератор *fresh-tail-gen* строит формулы для произвольного числа хвостов; измерения выполнены для $N = 1, \dots, 100$ без таймаутов.

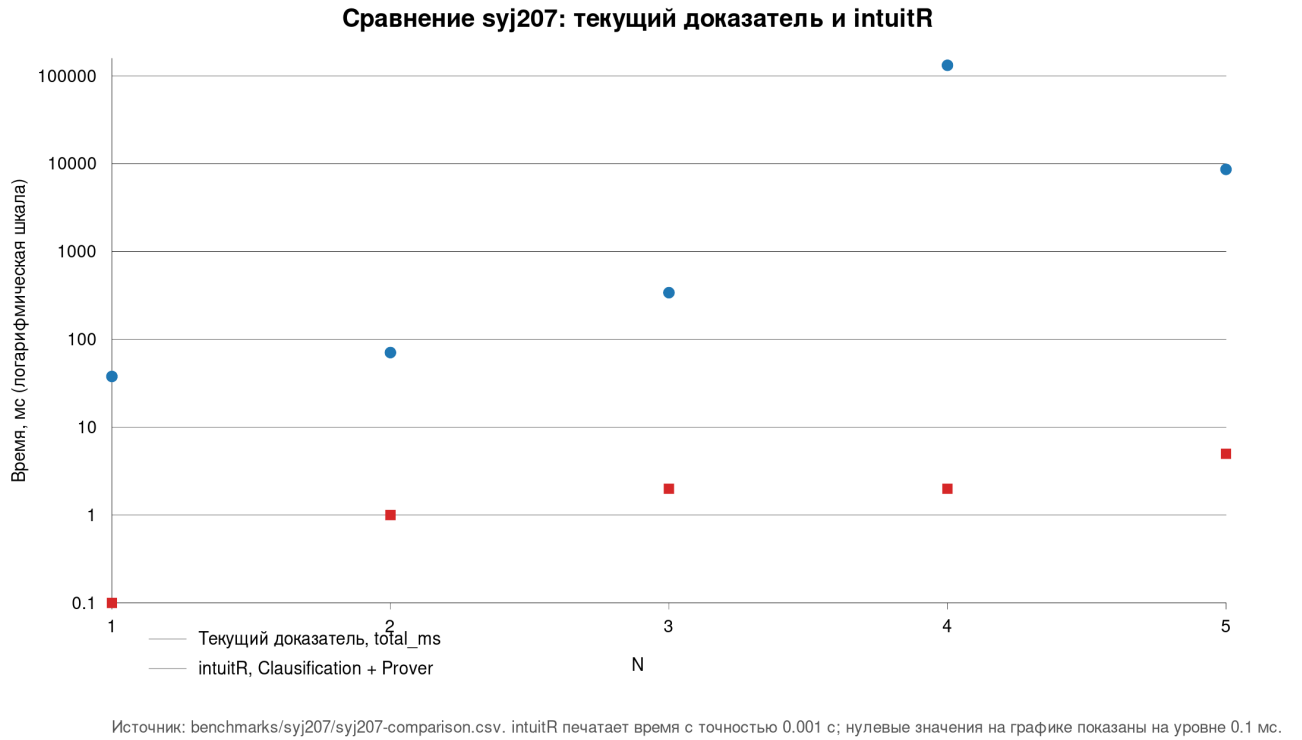


Рисунок 57 — Сравнение полного времени: прототип и intuitR на syj207

Таблица 5 — Время фаз прототипа на семействе fresh-tail (выборочные значения N)

N	Результат	Клауз., мс	Доказ., мс	Аннот., мс	Всего, мс
1	valid	1.244	68.710	0.236	70.191
10	valid	2.953	234.435	0.359	237.747
20	valid	2.693	397.140	0.358	400.191
30	valid	3.708	534.449	0.415	538.572
40	valid	4.695	878.023	0.448	883.166
50	valid	5.640	1427.600	0.493	1433.733
60	valid	7.764	3615.880	0.570	3624.214
70	valid	8.525	7416.072	0.704	7425.302
80	valid	11.394	3659.444	0.817	3671.656
90	valid	11.717	4929.499	0.821	4942.037
100	valid	15.130	4857.191	0.822	4873.143

Все сто формул доказаны успешно. Полное время растёт от 70 мс при $N = 1$ до 7,4 с при $N = 70$ (максимум серии) и 4,9 с при $N = 100$; рост не монотонен. Клауификация и аннотирование остаются пренебрежимо малыми; основной вклад по-прежнему даёт фаза доказательства.

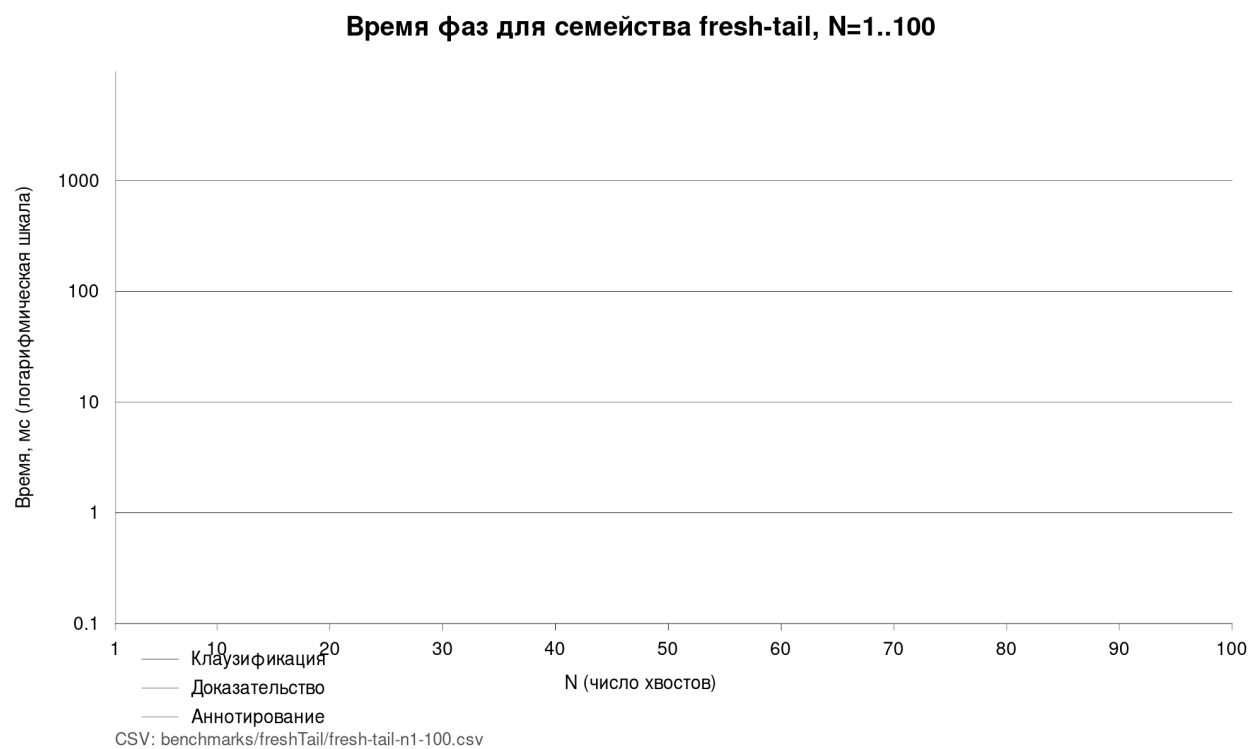


Рисунок 58 — Время фаз прототипа для fresh-tail, $N = 1, \dots, 100$

ЗАКЛЮЧЕНИЕ

В данной работе рассмотрена задача проверки выводимости формул интуиционистской пропозициональной логики и извлечения из найденных выводов программного содержания в виде λ -термов. Эта задача была рассмотрена с позиции изоморфизма Карри–Говарда, согласно которому доказательство формулы соответствует программе, а сама формула — типу этой программы.

В исследовательской части изложены теоретические основы: интуиционистская логика, ВНК-интерпретация, семантика Крипке, натуральный вывод, секвенциальное исчисление LJ и типизированное λ -исчисление. Показано, каким образом логические связки конъюнкции, дизъюнкции, импликации и абсурда получают вычислительную интерпретацию через произведения, суммы, функции и пустой тип.

В конструкторской части рассмотрены исчисления, применимые для автоматического поиска вывода: LJТ Дайкхоффа, SAT-ориентированное исчисление LJT_{SAT} и упрощённое исчисление $C^{\rightarrow}$ с процедурой перезапуска. Также описано, как клаузификация может быть представлена как последовательность допустимых правил секвенциального исчисления и как эти правила аннотируются λ -термами.

В технологической части разработан и описан программный прототип на Haskell. Актуальная реализация вынесена в пространство имён Refactoring и разделена на модули формул, клауз, секвенций, доказателя, λ -исчисления и визуализации. Конвейер прототипа включает клаузификацию исходной формулы, поиск вывода в $C^{\rightarrow}$ с использованием Z3 как инкрементального SAT/SMT-оракула, построение LJТ-поддеревьев для классических подзадач, аннотирование объединённого дерева λ -термами и вывод результата в формате Graphviz DOT. На примерах из каталога examples/formulas продемонстрированы выводимые формулы с редуцированными λ -термами и невыводимые формулы с конечными контрмоделями Крипке.

В аналитической части измерено время ключевых фаз конвейера на семействах бенчмарков syj201, syj207 и fresh-tail, а для syj201 и syj207 выполнено сравнение с аналогом intuitR. На выводимых формулах syj201 время прототипа растёт от 17 мс до 977 с, тогда как intuitR остаётся в пределах единиц миллисекунд. На невыводимых syj207 оба инструмента находят контрмодели, но

прототип может тратить минуты там, где `intuitR` укладывается в миллисекунды. Семейство `fresh-tail` (100 валидных формул) подтверждает работоспособность на глубокой вложенности импликаций с полным временем до 7,4 с. Основной вклад даёт фаза доказательства; среди причин разрыва с `intuitR` выделены асимптотика клаузификации и необходимое сохранение истории вывода для извлечения λ -терма и визуализации.

Практическим результатом работы является прототип, который для валидных формул строит дерево доказательства и редуцированный λ -терм, а для невалидных формул строит конечную контрмодель Крипке. Тем самым реализация связывает теоретическую часть работы с наглядным инструментом для изучения соответствия между формулами, доказательствами и программами.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Brouwer L. E. J. Collected Works. Volume 1: Philosophy and Foundations of Mathematics. — North-Holland, 1975.
2. Heyting A. Intuitionism: An Introduction. — North-Holland, 1956.
3. Kripke S. A. Semantical Analysis of Intuitionistic Logic I // Studies in Logic and the Foundations of Mathematics. — 1965. — T. 40. — C. 92—130.
4. Gentzen G. Untersuchungen über das logische Schließen // Mathematische Zeitschrift. — 1935. — T. 39. — C. 176—210, 405—431.
5. Claessen K., Rosén D. SAT modulo Intuitionistic Implications // Logic for Programming, Artificial Intelligence, and Reasoning (LPAR). T. 9450. — Springer, 2015. — C. 622—637. — (Lecture Notes in Computer Science).
6. Fiorentini C., Goré R., Graham-Lengrand S. A proof-theoretic perspective on SMT-solving for intuitionistic propositional logic // Automated Reasoning with Analytic Tableaux and Related Methods (TABLEAUX). T. 11714. — Springer, 2019. — C. 111—129. — (Lecture Notes in Computer Science).
7. Dyckhoff R. Contraction-free sequent calculi for intuitionistic logic // Journal of Symbolic Logic. — 1992. — T. 57, № 3. — C. 795—807.
8. Counterexample-Guided Abstraction Refinement / E. M. Clarke [и др.] // Computer Aided Verification (CAV). T. 1855. — Springer, 2000. — C. 154—169. — (Lecture Notes in Computer Science).
9. Fiorentini C. Efficient SAT-based Proof Search in Intuitionistic Propositional Logic // Automated Deduction (CADE). T. 12699. — Springer, 2021. — C. 217—233. — (Lecture Notes in Computer Science).

ПРИЛОЖЕНИЕ А

Листинг 48: Обратный поиск вывода в LJТ

```
1: function LJTProve( $\Gamma, G$ )
2:   if  $G$  — атом и  $G \in \Gamma$  then return true
3:   if  $\perp \in \Gamma$  then return true
4:   if  $\exists A \wedge B \in \Gamma$  then
5:      $\Gamma' \leftarrow \Gamma \setminus \{A \wedge B\}$  return LJTProve( $\Gamma' \cup \{A, B\}, G$ )
6:   if  $\exists A \vee B \in \Gamma$  then
7:      $\Gamma' \leftarrow \Gamma \setminus \{A \vee B\}$  return LJTProve( $\Gamma' \cup \{A\}, G$ )  $\wedge$  LJTProve( $\Gamma' \cup \{B\}, G$ )
8:   if  $\exists p \rightarrow B \in \Gamma, p$  — атом,  $p \in \Gamma$  then return LJTProve( $(\Gamma \setminus \{p \rightarrow B\}) \cup \{B\}, G$ )
9:   if  $\exists (A \wedge B) \rightarrow C \in \Gamma$  then
10:     $\Gamma' \leftarrow \Gamma \setminus \{(A \wedge B) \rightarrow C\}$  return LJTProve( $\Gamma' \cup \{A \rightarrow (B \rightarrow C)\}, G$ )
11:  if  $\exists (A \vee B) \rightarrow C \in \Gamma$  then
12:     $\Gamma' \leftarrow \Gamma \setminus \{(A \vee B) \rightarrow C\}$  return LJTProve( $\Gamma' \cup \{A \rightarrow C, B \rightarrow C\}, G$ )
13:  if  $G = A \rightarrow B$  then return LJTProve( $\Gamma \cup \{A\}, B$ )
14:  if  $G = A \wedge B$  then return LJTProve( $\Gamma, A$ )  $\wedge$  LJTProve( $\Gamma, B$ )
15:  if  $G = A \vee B$  then
16:    if LJTProve( $\Gamma, A$ ) then return true
17:    if LJTProve( $\Gamma, B$ ) then return true
18:  for  $(C_1 \rightarrow C_2) \rightarrow B \in \Gamma$  do
19:     $\Gamma' \leftarrow \Gamma \setminus \{(C_1 \rightarrow C_2) \rightarrow B\}$ 
20:    if LJTProve( $\Gamma' \cup \{C_2 \rightarrow B\}, C_1 \rightarrow C_2$ ) then return LJTProve( $\Gamma' \cup \{B\}, G$ )
21:  return false
```

ПРИЛОЖЕНИЕ Б

Вывод формулы $((((a \wedge b) \rightarrow c) \rightarrow ((a \rightarrow c) \vee (b \rightarrow c))) \rightarrow c) \rightarrow c$ в исчислении LJТ.

Для компактности обозначим $\alpha = (a \wedge b) \rightarrow c$ и $\beta = (a \rightarrow c) \vee (b \rightarrow c)$. Тогда доказываемая формула принимает вид $((\alpha \rightarrow \beta) \rightarrow c) \rightarrow c$.

$$\begin{array}{c}
 \frac{}{x:(a \rightarrow c) \rightarrow c, y:b \rightarrow c, z:a \vdash y : b \rightarrow c} Ax \\
 \frac{}{x:(a \rightarrow c) \rightarrow c, m:a \rightarrow (b \rightarrow c), z:a \vdash mz : b \rightarrow c} \rightarrow L_1 \quad \frac{}{x:(a \rightarrow c) \rightarrow c, k:c, m:a \rightarrow (b \rightarrow c) \vdash k : c} Ax \\
 \frac{}{x:(a \rightarrow c) \rightarrow c, p:(b \rightarrow c) \rightarrow c, m:a \rightarrow (b \rightarrow c), z:a \vdash p(mz) : c} \rightarrow R \\
 \frac{}{x:(a \rightarrow c) \rightarrow c, p:(b \rightarrow c) \rightarrow c, m:a \rightarrow (b \rightarrow c) \vdash \lambda z. p(mz) : a \rightarrow c} \rightarrow L_2 \\
 \frac{}{x:(a \rightarrow c) \rightarrow c, p:(b \rightarrow c) \rightarrow c, q:\alpha \vdash \lambda z. p(\lambda w. q\langle z, w \rangle) : a \rightarrow c} \rightarrow L_3 \\
 \frac{}{s:\beta \rightarrow c, q:\alpha \vdash \lambda z. (\lambda u. s(Ru))(\lambda w. q\langle z, w \rangle) : a \rightarrow c} \vee R_1 \\
 \frac{}{s:\beta \rightarrow c, q:\alpha \vdash L(\lambda z. (\lambda u. s(Ru))(\lambda w. q\langle z, w \rangle)) : \beta} \rightarrow R \\
 \frac{}{s:\beta \rightarrow c \vdash \lambda q. L(\lambda z. (\lambda u. s(Ru))(\lambda w. q\langle z, w \rangle)) : \alpha \rightarrow \beta} \rightarrow L_4 \quad \frac{}{k:c \vdash k : c} Ax \\
 \frac{}{r:(\alpha \rightarrow \beta) \rightarrow c \vdash r(\lambda q. L(\lambda z. (\lambda u. (\lambda t. r(Kt))(Ru))(\lambda w. q\langle z, w \rangle))) : c} \rightarrow R \\
 \vdash \lambda r. r(\lambda q. L(\lambda z. (\lambda u. (\lambda t. r(Kt))(Ru))(\lambda w. q\langle z, w \rangle))) : ((\alpha \rightarrow \beta) \rightarrow c) \rightarrow c
 \end{array}$$

ПРИЛОЖЕНИЕ В

Ниже приведены процедуры поиска вывода в исчислении LJT_{SAT} , основанные на работе Фиорентини, Горе и Грэма-Ленгрена. Процедуры используют инкрементальный SAT-решатель s с интерфейсом:

- $\text{newSolver}()$ — создать новый SAT-решатель;
- $\text{addClause}(s, \varphi)$ — добавить плоскую клаузу φ в решатель s ;
- $\text{satProve}(s, A, q)$ — проверить, выполняется ли $R(s)$, $A \vdash_{cpl} q$, где $R(s)$ — множество клауз в s , A — множество атомарных допущений. Возвращает $\text{Yes}(A')$ с $A' \subseteq A$ или $\text{No}(M)$ с классической контрмоделью M .

Листинг 49: Главная процедура LJT_{SAT}

```
1: function LJTSatMain( $R_0, X_0, q_0$ )
2:    $s \leftarrow \text{newSolver}()$ 
3:   for  $\varphi \in R_0$  do
4:      $\text{addClause}(s, \varphi)$ 
5:   for  $(a \rightarrow b) \rightarrow c \in X_0$  do
6:      $\text{addClause}(s, b \rightarrow c)$ 
7:    $\tau \leftarrow \text{LJTSat}(R_0, X_0, \emptyset, q_0)$ 
8:   if  $\tau = \text{No}(K)$  then
9:     return  $K$ 
10:  else
11:    return вывод в  $LJT_{SAT}$ 
```

Параметр r (мир контрмодели) опущен для краткости; при построении контрмодели миры индексируются последовательностями импликационных клауз, образующими дерево вызовов.

Листинг 50: Рекурсивная процедура поиска вывода

```

1: function LJTSat( $R, X, A, q$ )
2:    $\tau_0 \leftarrow \text{satProve}(s, A, q)$ 
3:   if  $\tau_0 = \text{Yes}(A')$  then
4:     return  $\text{Yes}(A')$ 
5:    $M \leftarrow$  контрмодель из  $\tau_0$ 
6:    $\Theta \leftarrow \emptyset$ 
7:   for  $\iota = (a \rightarrow b) \rightarrow c \in X$  т. ч.  $a \notin M, b \notin M, c \notin M$  do
8:      $X_\iota \leftarrow X \setminus \{\iota\}$ 
9:      $\tau_1 \leftarrow \text{LJTSat}(R \cup \{b \rightarrow c\}, X_\iota, M \cup \{a\}, b)$ 
10:    if  $\tau_1 = \text{No}(K_1)$  then
11:       $\Theta \leftarrow \Theta \cup \{(\iota, K_1)\}$ 
12:    else
13:       $A_1 \leftarrow$  множество атомов из  $\tau_1$ 
14:       $\tilde{\varphi} \leftarrow \bigwedge (A_1 \setminus \{a\}) \rightarrow c$ 
15:       $\text{addClause}(s, \tilde{\varphi})$ 
16:       $\tau_2 \leftarrow \text{LJTSat}(R \cup \{\tilde{\varphi}\}, X, A, q)$ 
17:      if  $\tau_2 = \text{No}(K_2)$  then
18:        return  $\text{No}(K_2)$ 
19:      else
20:        return  $\tau_2$ 
21:  return  $\text{No}(\text{Mod}(r, \Theta, M))$ 

```

ПРИЛОЖЕНИЕ Г

Ниже приведён основной листинг функции `prove`, которая связывает клаузацию, запуск SAT-based алгоритма, расширение доказательства LJT-поддеревьями, аннотирование λ -термами, редукцию и вывод результата в формате DOT.

Листинг 51 показывает, как функция `prove` последовательно строит все промежуточные представления, печатает промежуточное дерево клаузации и в конце выводит DOT-граф доказательства или контрмодель.

Листинг 51: Функция prove: общий конвейер доказательства

```

1 prove :: Formula_ -> IO ()
2 prove formula = do
3   let goalAtom = Variable "$"
4   let goal = Atom goalAtom
5   let sequent = Unclassified [] [] [Annotated () (implication formula
6     goal)] (Annotated (), ()) goalAtom)
7   let (clausified, st) = runState (clausify sequent) (ClausificationState
8     0)
9   fmtLn $ buildDotClausify 0 clausified
10
11   let (Intuit flats impls goal) = getSequent clausified
12   s@(Solver context solver _) <- newSolver
13
14   let universe = Set.toList $ Set.fromList $ annotated goal :
15     (atoms =<< (map (toFormula . annotated) flats ++ map (toFormula .
16       annotated) impls))
17
18   universeAsts <- mapM (mkFreshAtom context) universe
19   let atomToAst = Map.fromList $ zip universe universeAsts
20
21   let s = Solver context solver [Annotated (atomToAst ! a) a | a <-
22     universe]
23   let reannotatedFlats = Annotated () <$> reannotateFlat (atomToAst !)
24     <$> annotated <$> flats
25   mapM_ (addClause s) (annotated <$> reannotatedFlats)
26
27   let reannotatedImpls = Annotated () <$> reannotateImpl (atomToAst !)
28     <$> annotated <$> impls
29   let reannotatedGoal = reannotate ((((), ) . (atomToAst !)) goal
30   let reannotatedIntuit = Intuit reannotatedFlats reannotatedImpls
31     reannotatedGoal
32
33   result <- carrow reannotatedIntuit s
34   case result of
35     Left counterModel -> fmtLn $ counterModel |+ ""
36     Right proof -> do
37       let extended = extendProof (second (const ())) proof)
38       let concated = concatTrees clausified (second (const ())) extended)
39       let (annotatedProof, _) = runState (annotateClausification
40         concated) (Environment 0 Map.empty)
41       let reducedAnnotatedProof = first Lambda.reduce annotatedProof
42       fmtLn $ "digraph {\n" <>
43         "  graph [rankdir=BT]\n" <>
44         "  node [shape=record;fontname=Arial]\n" <>
45         indentF 2 (buildDotClausify 0 reducedAnnotatedProof) <>
46         "}\n"
47
48   return ()

```

ПРИЛОЖЕНИЕ Д

Ниже приведён псевдокод процедуры `proveR`, соответствующий схеме из работы Фиорентини [9]. Процедура принимает r -секвенцию $R, X \Rightarrow g$ и возвращает один из двух результатов: `Valid`, если секвенция выводима, или `CountSat`, если построена контрмодель Крипке.

Листинг 52: Процедура `proveR` с перезапуском

```
1: function proveR( $R, X, g$ )
2:    $s \leftarrow \text{newSolver}(R)$ 
3:   loop
4:      $W \leftarrow \emptyset$ 
5:      $\tau \leftarrow \text{satProve}(s, \emptyset, g)$ 
6:     if  $\tau = \text{Yes}(\emptyset)$  then
7:       return Valid
8:     Пусть  $\tau = \text{No}(M)$ 
9:     loop
10:       $W \leftarrow W \cup \{M\}$ 
11:      Найти пару  $\langle w, \lambda \rangle$  такую, что  $w \in W, \lambda \in X$  и  $w \not\models_W \lambda$ 
12:      if такой пары нет then
13:        return CountSat
14:      Пусть  $\lambda = (a \rightarrow b) \rightarrow c$ 
15:       $\tau \leftarrow \text{satProve}(s, w \cup \{a\}, b)$ 
16:      if  $\tau = \text{No}(M')$  then
17:         $M \leftarrow M'$ 
18:      else
19:        Пусть  $\tau = \text{Yes}(A)$ 
20:         $\varphi \leftarrow \bigwedge(A \setminus \{a\}) \rightarrow c$ 
21:         $\text{addClause}(s, \varphi)$ 
22:        break
```

Внешний цикл процедуры соответствует последовательному расширению множества плоских клауз $R(s)$ выученными клаузами. Внутренний цикл строит множество интерпретаций W , которое при невозможности выбрать нарушенную импликационную клаузу становится контрмоделью. Если же такая клауза найдена и проверка $R(s), w, a \vdash_{cpl} b$ завершается успехом, процедура извлекает множество допущений A , строит выученную клаузу φ и перезапускает главный цикл.

ПРИЛОЖЕНИЕ Е

Ниже приведены схемы аксиом гильбертовой системы для интуиционистской пропозициональной логики и правило modus ponens.

$$\frac{A \quad A \rightarrow B}{B}$$

Рисунок 59 — Правило modus ponens

- (A1) $A \rightarrow (B \rightarrow A)$
- (A2) $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$
- (A3) $A \wedge B \rightarrow A$
- (A4) $A \wedge B \rightarrow B$
- (A5) $A \rightarrow (B \rightarrow A \wedge B)$
- (A6) $A \rightarrow A \vee B$
- (A7) $B \rightarrow A \vee B$
- (A8) $(A \rightarrow C) \rightarrow ((B \rightarrow C) \rightarrow (A \vee B \rightarrow C))$
- (A9) $\perp \rightarrow A$

Рисунок 60 — Аксиомы гильбертовой системы для IPL